

Multiplication, exponentiation and factorization of big numbers

Pavel Atnashev

patnashev@gmail.com

September 25, 2023

Introduction

Since the invention of simple counting and natural numbers, people are dealing with a phenomenon called big numbers. Most languages have special words to designate numbers larger than ten: a dozen, a hundred, a thousand. When that became not enough, new words like ‘million’, ‘milliard’, ‘billion’, ‘trillion’ were invented, although those words may mean different numbers in different cultures. For even larger numbers a special notation based on powers of 10 is widely used in sciences. One gram of ordinary matter contains approximately 10^{24} nucleons (Avogadro number). The estimated number of nucleons in the observable universe is 10^{80} (Eddington number).

It may seem that numbers exceeding the amount of matter in the Universe are useless. But they arise naturally in various branches of mathematics and could be useful in practice, for example in cryptography. Modern cryptography deals with numbers from 100 to 1000 decimal digits long. Despite the formidable size of such numbers, modern computing devices can perform basic arithmetic operations on them with ease.

But human curiosity has no bounds. In areas of study such as experimental mathematics and computational number theory scientists formulate questions and find answers about properties of numbers with millions of decimal digits. At the time of writing the largest known prime number has 24 862 048 digits. Arithmetic operations on such numbers take just a few milliseconds on modern computers thanks to algorithms specifically developed for this task.

However, mathematics knows nontrivial numbers so big that even the simplest arithmetic operations are not possible on them. The largest known Carmichael number has 295 486 761 787 digits. The largest Fermat number with a known divisor has $10^{5488968}$ digits. All atoms in the Universe are not enough to simply write that number in decimal notation. But we know that it is divisible by a known, comprehensible prime number.

This work considers numbers to be ‘big’ if they have from 1 thousand to 100 million decimal digits. To store such numbers one needs from a few kilobytes to tens of megabytes of system memory or disk space. Addition and subtraction operations are linear and present no difficulty in most cases. Most difficulties arise with operations of multiplication and multiplication modulo another big number. Those two operations are the subjects of the first chapter of this work. The second chapter is devoted to raising big numbers to big powers and checking the correctness of the computation. In the third chapter we investigate the ways to find prime factors of big numbers.

Contents

1	Multiplication	3
1.1	Representation	3
1.2	Multiplication	4
1.3	Fast multiplication	5
1.4	Fourier transform	7
1.5	Complex field	8
1.6	Finite field	10
1.7	Hierarchical memory	10
1.8	Modular multiplication	12
1.9	Weighted transform	13
1.10	Free word space	16
1.11	Nested multiplication	17
1.12	Implementation	19
2	Exponentiation	21
2.1	Binary exponent	21
2.2	Gerbicz check	22
2.3	Atnashev certificate	24
2.4	Pietrzak function	26
2.5	Non-binary exponent	28
2.6	Li generalization	30
2.7	Lucas sequences	32
2.8	Edwards curves	35
3	Factorization	40

Chapter 1

Multiplication

1.1 Representation

High-performance computing devices are most efficient dealing with data types having fixed size. To reach the optimal performance of operations, big numbers have to be represented by many smaller individual elements, the size of which is optimal for the specific computing device. Modern central processors have specialized vector units which can perform arithmetic operations on several floating-point numbers at once, which makes this data type preferable. On the other hand, graphical (parallel) processors are optimized for performing operations on streams of short integer data. They need their own algorithms to be efficient. This leads to a situation when the same task may be executed by different algorithms using completely different representations, depending on the available computing device.

The most convenient way to represent a big number is to write it as a polynomial over field $\mathbb{F}$, which takes the value equal to the big number at a fixed point which is called *base*. Let's designate the big number as $\mathbf{a}$, corresponding polynomial as $A(x)$, its coefficients as a_i , its degree as $N - 1$. Then

$$\mathbf{a} = A(\text{base}) = \sum_{i=0}^{N-1} a_i \text{base}^i, a_i \in \mathbb{F}, \text{base} \in \mathbb{F} \quad (1.1.1)$$

The decimal numeral system is an example of such representation. It uses base 10, and each possible value of coefficients has its own symbol called 'digit'.

$$3196 = [3x^3 + 1x^2 + 9x + 6](10) = 3 \cdot 1000 + 1 \cdot 100 + 9 \cdot 10 + 6$$

It is worth noting that the representation itself does not depend on the numeral system we use to obtain it. In hexadecimal numeral system the representation of number C7C in base A is still 3 1 9 6:

$$\text{C7C}_{16} = [3x^3 + 1x^2 + 9x + 6](A_{16}) = 3_{16} \cdot 3E8_{16} + 1_{16} \cdot 64_{16} + 9_{16} \cdot A_{16} + 6_{16}$$

For larger bases coefficients of the polynomial are often called 'words', less often 'digits' (not to be confused with decimal digits). Unlike numeral systems, representations do not subject words to strict bounds like $0 \leq a_i < \text{base}$. There are algorithms efficiently dealing with negative words, irrational words, words with absolute value larger than the square of base. The only strict requirement is that $A(\text{base}) = \mathbf{a}$. Obviously, this makes representations not unique, with several representations equal to the same big number. This fact is actually used in many multiplication algorithms in the so-called 'normalization' stage, when one representation gets replaced with another one, more suitable for following operations.

Almost all modern computing devices use binary numeral system, which makes a power of two the logical choice for the base. It makes storing big numbers more efficient, speeds up construction of a representation and simplifies normalization. But in some cases such a base is not the best possible choice. For example,

for operations modulo $b^n + c$ sometimes it is more beneficial to use base b or even $\sqrt{b}$. Constructing such a representation becomes a nontrivial but solvable task.

1.2 Multiplication

Let $\mathbf{a}$ and $\mathbf{b}$ be big numbers, $A(x)$ and $B(x)$ their corresponding polynomials. The product $\mathbf{a} \cdot \mathbf{b}$ can be computed as a product of two polynomials:

$$P(x) = A(x) \times B(x) \quad (1.2.1)$$

$$\mathbf{a} \cdot \mathbf{b} = P(\text{base}) \quad (1.2.2)$$

The simplest way to compute $P(x)$ is the well-known to everyone ‘grade-school multiplication’ method:

$$p_k = \sum_{i+j=k} a_i b_j \quad (1.2.3)$$

This method is not only simple, but also very efficient. For small N it runs faster than more complicated methods because of no overhead. Besides, it can also be used to debug other methods even for large N . Computing of an individual p_k takes linear time and can be used for “quality control” of a more complicated multiplication algorithm. To perform such a control operation a random k is chosen, and p_k computed by different methods are compared.

It is important to properly estimate the absolute value of p_k with the goal to choose the suitable data type to store the coefficients. If both input polynomials have the same degree, then $N - 1$ -th coefficient is

$$p_{N-1} = a_0 b_{N-1} + a_1 b_{N-2} + \dots + a_{N-2} b_1 + a_{N-1} b_0,$$

its absolute value can reach $N \cdot \text{base}^2$ on condition that all a_i and b_j are less than base . But if $P(x)$ is used again as an input polynomial in a subsequent multiplication operation, then the resulting coefficients will be even larger. To avoid uncontrolled growth of coefficients, a ‘normalization’ operation is used to reduce coefficients to the normal form determined by the representation. This operation is also known as ‘carry propagation’.

Suppose the normal form is set as $0 \leq a_i < \text{base}$, then the normalization can be described as finding of a new polynomial $P^{\text{norm}}(x)$ such that

$$P^{\text{norm}}(\text{base}) = P(\text{base}), 0 \leq p_k^{\text{norm}} < \text{base} \quad (1.2.4)$$

It’s important to understand that the polynomial $P^{\text{norm}}(x)$ is not equal to the polynomial $P(x)$, they can even have different degrees. But they take the same value at base point. Therefore, the normalization operation is bound to the specific value of base , and the presence of this operation makes multiplication of big numbers different from multiplication of polynomials.

Also, the normalization stage is a good place to perform small additional linear operations. For example, multiplication by a small constant like 2 or -1, addition of a small constant to the result and such.

The implementation of normalization totally depends on the representation, but in general terms it looks like a grade-school addition with carry propagation. Each subsequent coefficient gets carry added to it, the result is compared to the base, the reduced coefficient is calculated and the next carry is formed. Let’s take an example. In the process of squaring number 3196 in the decimal representation, one gets the following polynomial after applying (1.2.3):

$$P(x) = 9x^6 + 6x^5 + 55x^4 + 54x^3 + 93x^2 + 108x + 36$$

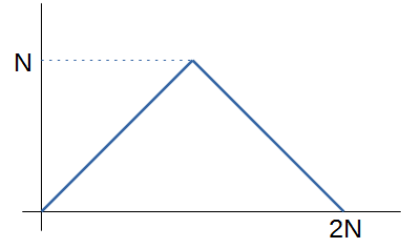


Figure 1.1: Number of terms in p_k .

After normalization one gets the polynomial of a higher degree:

$$P^{norm}(x) = 1x^7 + 0x^6 + 2x^5 + 1x^4 + 4x^3 + 4x^2 + 1x + 6$$

But both polynomials take the same value at point 10:

$$P^{norm}(10) = P(10) = 10214416$$

1.3 Fast multiplication

The grade-school multiplication method has asymptotic complexity of $O(N^2)$. It was considered the best theoretically possible algorithm for a long time, until Anatoly Karatsuba discovered a faster method in 1960. Karatsuba has become a like of Lobachevsky of computational complexity. After he had broken the “square wall”, several even faster algorithms have been discovered.

Karatsuba method is a recursive algorithm on each step of which each multiplier is divided into two halves. But instead of four multiplications only three multiplications are performed, with a few extra additions and subtractions.

$$\{a, b\} \times \{c, d\} = \{ac, ad + bc, bd\} = \{ac, (a + b)(c + d) - ac - bd, bd\} \quad (1.3.1)$$

The algorithm has asymptotic complexity of $O(N^{\log_2 3})$, $\log_2 3 = 1, 58\dots$

There are algorithms with asymptotic complexity of $O(N \log N \log \log N)$. In a sense they can be considered running in (pseudo)linear time. Only thanks to those algorithms the search for prime numbers with tens of millions of decimal digits is possible. Such algorithms are commonly called ‘fast’.

Fast algorithms are based on the following property of polynomials:

$$\forall r \in \mathbb{F}: P(r) = A(r) \cdot B(r) \quad (1.3.2)$$

Instead of computing coefficients of polynomial $P(x)$, one computes its values at a set of distinct points $\{r_i\}$. It can be done by computing values of polynomials $A(x)$ and $B(x)$ at those points and then multiplying them. The coefficients of polynomial $P(x)$ can be reconstructed with the help of another property of polynomials known as the little Bezout’s theorem:

$$P(r) = P(x) \bmod (x - r) \quad (1.3.3)$$

Note that the modulo operation on polynomials can be easily implemented with the long division algorithm.

Thus, we can declare $\{P(r_i)\}$ a set of degree-0 polynomials, each of which is a remainder of polynomial $P(x)$ modulo one of the pairwise coprime polynomials $x - r_i$. To find remainders modulo higher degree polynomials, one can successively apply the Chinese remainder theorem, which for arbitrary polynomial moduli $R(x)$ and $Q(x)$ can be formulated as:

$$\begin{aligned} P(x) \bmod (R(x) \times Q(x)) = & ((P(x) \bmod R(x)) \times Q(x)^{-1} \bmod R(x)) \times Q(x) \\ & + ((P(x) \bmod Q(x)) \times R(x)^{-1} \bmod Q(x)) \times R(x) \end{aligned} \quad (1.3.4)$$

The process continues until only one polynomial remains:

$$P_N(x) = P(x) \bmod \prod_{i=0}^{N-1} x - r_i \quad (1.3.5)$$

The degree of the polynomial is not exceeding $N - 1$. By choosing N big enough so that $N > \deg(A) + \deg(B)$ one can be sure that $P_N(x) = P(x)$.

For optimal performance, one should apply (1.3.4) to pairs of polynomials of the same degree. As a result, after the first step one gets $N/2$ degree-1 polynomials, after the second step $N/4$ degree-3 polynomials, and so on until after the $\log N$ -th step only one degree- $N - 1$ polynomial remains. Also, one should remember

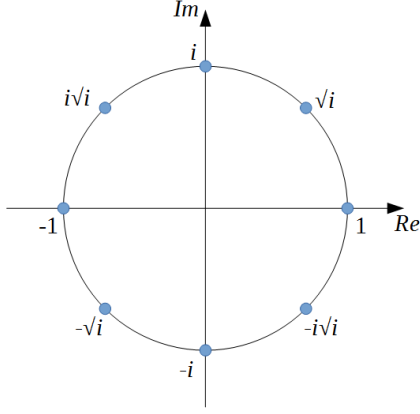


Figure 1.2: Roots of $x^8 - 1$.

that (1.3.4) relies on several polynomial multiplications, making the algorithm recursive. Its asymptotic complexity can be estimated as $O(N \log^2 N)$, which is somewhat worse than stated above. But we can further improve the algorithm.

The thing is, we're not constrained in any way in our choice of the set of points $\{r_i\}$. We're free to choose such points that are convenient for us in subsequent computations. The most popular choice is the roots of polynomial $x^N - 1$ (fig. 1.2). Let's assume that N is a power of two. Such a set of points has a very useful property: for each point r the set also contains $-r$ and r^2 . Let's sort the points by their order, making sure that $-r$ is next to r :

$$\{r_i\} = 1, -1, i, -i, \sqrt{i}, -\sqrt{i}, i\sqrt{i}, -i\sqrt{i}, \dots \quad (1.3.6)$$

This gives us a binary tree of corresponding polynomials:

$$\begin{aligned} (x-1) \times (x+1) &= x^2 - 1 \\ (x-i) \times (x+i) &= x^2 + 1, \quad (x^2-1) \times (x^2+1) = x^4 - 1 \\ (x-\sqrt{i}) \times (x+\sqrt{i}) &= x^2 - i \\ (x-i\sqrt{i}) \times (x+i\sqrt{i}) &= x^2 + i, \quad (x^2-i) \times (x^2+i) = x^4 + 1, \quad (x^4-1) \times (x^4+1) = x^8 - 1 \end{aligned}$$

Easy to see that each level of the tree is composed of polynomials of $x^n - r$ form, where n is a power of two and r is one of the first N/n elements of sequence (1.3.6). Let's apply the tree to (1.3.4), assuming $R(x) = x^n - r$, $Q(x) = x^n + r$, $P(x) \bmod R(x) = P^+(x)$, $P(x) \bmod Q(x) = P^-(x)$:

$$\begin{aligned} P(x) \bmod (x^{2n} - r^2) &= (P^+(x) \times (x^n + r)^{-1} \bmod (x^n - r)) \times (x^n + r) \\ &\quad + (P^-(x) \times (x^n - r)^{-1} \bmod (x^n + r)) \times (x^n - r) \\ &= (P^+(x) \cdot \frac{1}{2r}) \times (x^n + r) + (P^-(x) \cdot \frac{-1}{2r}) \times (x^n - r) \\ &= \frac{P^+(x) + P^-(x)}{2} + \frac{P^+(x) - P^-(x)}{2r} \times x^n \end{aligned} \quad (1.3.7)$$

It's popular in software implementations of this algorithm to store coefficients of $P^+(x)$ and $P^-(x)$ in the same data array, so that the coefficients of $P^-(x)$ look like p_{n+k} . In such a case (1.3.7) turns into:

$$p'_k = \frac{p_k + p_{n+k}}{2}, \quad p'_{n+k} = \frac{p_k - p_{n+k}}{2r} \quad (1.3.8)$$

Usually $\frac{1}{2}$ is factored out on each step, while multiplying the result by $\frac{1}{N}$ in the end.

Thanks to the good choice of the set $\{r_i\}$, we've reduced the recursive multiplication of polynomials to a linear operation consisting of addition, subtraction and multiplication by an inverse. The same can be done with the computation of values of polynomials $A(x)$ and $B(x)$ at $\{r_i\}$ points. The following property of the modulo operation will help us:

$$A(x) \bmod R(x) = A(x) \bmod (R(x) \times Q(x)) \bmod R(x) \quad (1.3.9)$$

In the binary tree of polynomials each node divides all its ascendants, so we can use a remainder modulo any ascendant to calculate the remainder modulo the node. At the top, polynomial $A(x)$ is declared to be equal to $A(x) \bmod (x^N - 1)$, then two moduli $x^{N/2} - 1$ and $x^{N/2} + 1$ are applied to produce descendant polynomials. We can do it because those moduli divide $x^N - 1$. This operation is repeated recursively, halving the degree of polynomials on each step, until we get degree-0 polynomials, which according to the little Bezout's theorem are equal to the values of $A(x)$ at the roots of $x^N - 1$.

We get polynomials of $A(x) \bmod (x^{2n} - r^2)$ form as the input on each step of the recursion. Our task is to compute polynomials $A^+(x)$ and $A^-(x)$ such as

$$A^+(x) = A(x) \bmod (x^n - r), \quad A^-(x) = A(x) \bmod (x^n + r) \quad (1.3.10)$$

Once again, it's convenient to store coefficients of $A^+(x)$ and $A^-(x)$ in a single data array, the coefficients of $A^-(x)$ as a'_{n+j} . After performing the long division, we get:

$$a'_j = a_j + a_{n+j}r, \quad a'_{n+j} = a_j - a_{n+j}r \quad (1.3.11)$$

This operation also consists only of addition, subtraction and multiplication. On the next step of the recursion we get $A^+(x)$ as input and use polynomials $x^{n/2} - \sqrt{r}$ and $x^{n/2} + \sqrt{r}$, which divide $x^n - r$. We also get $A^-(x)$ and use polynomials $x^{n/2} - i\sqrt{r}$ and $x^{n/2} + i\sqrt{r}$. In general, the constant terms of polynomial moduli used on each step are the first N/n elements of the sequence (1.3.6). In fact, such a recursion can even be used to construct the sequence (1.3.6) in the first place.

It's easy to see that operations (1.3.11) and (1.3.8) are inverses of each other.

1.4 Fourier transform

Experienced reader may have noticed that operations (1.3.11) and (1.3.8) are also known as the forward and the inverse fast Fourier transforms (FFT). It is indeed so, and it's not hard to understand why.

FFT is a fast algorithm computing the discrete Fourier transform (DFT), which is defined as

$$\bar{a}_k = \sum_{j=0}^{N-1} a_j e^{-i \frac{2\pi}{N} jk} \quad (1.4.1)$$

Here and later the line over a symbol means transformed data. It's easy to see that each coefficient a_j is multiplied by the same value raised to j -th power. It is equivalent to computing the value of polynomial $A(x)$ at that point:

$$\bar{a}_k = A(e^{-i \frac{2\pi}{N} k}) \quad (1.4.2)$$

Points $e^{-i \frac{2\pi}{N} k}$ are uniformly distributed on the unit circle in the complex plane. Each of them is equal to 1 when raised to N -th power. In other words, the points are the roots of equation $x^N - 1 = 0$. Therefore, the discrete Fourier transform by definition is equivalent to evaluation of polynomial $A(x)$ at the roots of polynomial $x^N - 1$, exactly the same thing as we do in the previous section.

In practice it means that all fast algorithms developed for computation of DFT can be used for multiplication of big numbers, including algorithms with N not equal to a power of two.

At the same time, there's no need to be an expert on the properties of Fourier transform to successfully multiply big numbers. On the contrary, it may be harmful because it limits the perception only to the issues specifically related to DFT. But DFT is not the only possible transform. For example, it is popular in practice to use roots of polynomial $x^N + 1$ (fig. 1.3) as the set $\{r_i\}$. The algorithm stays fast, but it is not FFT anymore, it is not even a Fourier transform anymore. Attempts to explain how it works in terms of discrete transforms may lead to unnecessary confusion and a false sense that it's some kind of a new unrelated algorithm.

To make matters worse, the $x^N + 1$ algorithm has no proper name. One may come across names like 'the negative half of FFT', 'negacyclic convolution', 'z-transform with $z^N = -1$ ', 'chirp z-transform', 'weighted

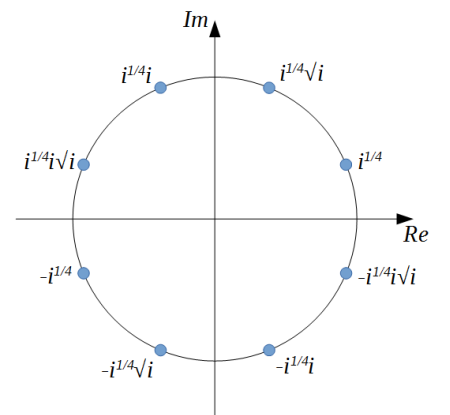


Figure 1.3: Roots of $x^8 + 1$.

transform with $\sqrt[N]{-1}$ weight', 'evaluation of polynomial at the roots of $x^N + 1$ '. And, the worst of all, some software libraries implementing the algorithm are calling it 'FFT' for historical reasons. On the other hand, while the algorithm is not FFT, it is very similar to FFT in terms of software implementation, leading to the confusion.

Operations (1.3.11) and (1.3.8) stay the same in an implementation of $x^N + 1$ algorithm, the only difference is the choice of r on each step of the recursion. After multiplying each point by $\sqrt[N]{-1}$ ("rotating" by the half angle between them), we get the following tree of polynomials:

$$\begin{aligned} (x - \sqrt[8]{-1}) \times (x + \sqrt[8]{-1}) &= x^2 - \sqrt{i} \\ (x - \sqrt[8]{-1}i) \times (x + \sqrt[8]{-1}i) &= x^2 + \sqrt{i}, \quad (x^2 - \sqrt{i}) \times (x^2 + \sqrt{i}) = x^4 - i \\ (x - \sqrt[8]{-1}\sqrt{i}) \times (x + \sqrt[8]{-1}\sqrt{i}) &= x^2 - i\sqrt{i} \\ (x - \sqrt[8]{-1}i\sqrt{i}) \times (x + \sqrt[8]{-1}i\sqrt{i}) &= x^2 + i\sqrt{i}, \quad (x^2 - i\sqrt{i}) \times (x^2 + i\sqrt{i}) = x^4 + i, \quad (x^4 - i) \times (x^4 + i) = x^8 + 1 \end{aligned}$$

It is worth noting that the roots used here are also used in FFT twice the size, FFT size 16 for the example above. That is the reason for the name 'the negative half of FFT'. But contrary to FFT, each root is used only once in the whole tree of polynomials.

1.5 Complex field

The most obvious choice for the field of coefficients is the complex field $\mathbb{F} = \mathbb{C}$, which can be implemented with double-precision floating-point numbers (double, FP64). Operations on this data type can be performed very efficiently thanks to vector instructions (SSE, AVX, AVX-512) supported by modern central processors. Such instructions can perform a single operation on up to 8 values at the same time. Also, there are additional useful instructions, like fused multiply-add (FMA), which can perform several operations with a single rounding.

The performance of an algorithm depends, first of all, on the amount of operations. When choosing the representation of a big number, the goal is to have the smallest N possible, and therefore the largest base possible. As already noted, the limiting factor is the ability of the data type to store coefficients of the result. Integers up to 2^{53} can be stored in a variable of FP64 type without the loss of the lowest bits. In case even a single coefficient of the result exceeds this limit, its value becomes inaccurate, and the result as a whole becomes incorrect. Such a situation is called 'roundoff error'. Issues related to roundoff errors demand constant attention in practice.

Of course, we can estimate the worst possible outcome and choose N big enough to have a guarantee that we never exceed data type limits. Such approach may demand more operations to be performed than necessary, but it can be justified when an absolute guarantee of correctness is required. For example, when performing a quick check of the results of some algorithm. But for the bulk of calculations a more subtle approach is desired.

One of the main difficulties related to roundoff errors comes from the fact that they're not very well understood theoretically. Most of the following things come from empirical observations.

As an estimate of the level of error we use the maximum deviation of a coefficient from integer value. It obviously can't exceed 0.5, otherwise we wouldn't be able to distinguish two close integer values. Besides, it's a good practice to define some guard level S , crossing which would indicate that we're dangerously close to the level of errors. Often, 0.4 is selected as the guard level.

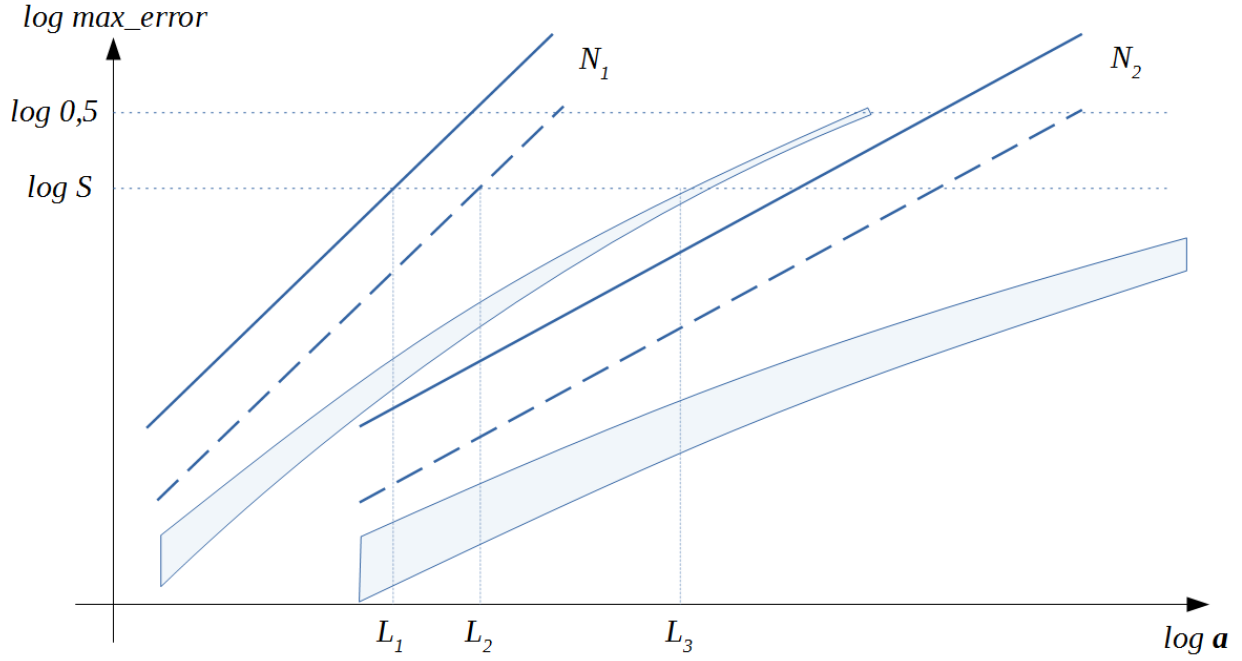


Figure 1.4: Roundoff errors of two transforms of N_1 and N_2 sizes. Solid line corresponds to the theoretical bound. Dotted line corresponds to the maximum level observed in practice for “unlucky” numbers. Shaded area corresponds to roundoff errors of random inputs.

Fig. 1.4 shows an outline of roundoff error levels expected during multiplication of big numbers of various sizes using two fixed transforms. When the size of big numbers grow, the base of representation grows too, which leads to growing roundoff errors. At some moment the choice of transform size N_1 becomes too dangerous because error level reaches S . At this moment one should transition to transform size N_2 . But there are several strategies of choosing the moment when the transform size is changed. Transitioning at the moment L_1 , one gets guarantees proved by the theoretical estimates that all performed computations are correct. Transitioning at the moment L_2 , one avoids all known pathological cases. But for most operations, when input numbers look “random”, are not powers of two, do not contain repeating bit sequences, are not a simple function of the base, then it is quite reasonable to use the first transform up to the moment L_3 .

But what should one do if they’re sure that an input is not “random”? For example, in the beginning or the end of Fermat primality test. Any multiplication can be made ‘safe’ with the help of a random big number r :

$$a \cdot b = (a + r) \cdot (b - r) + r \cdot (a + r - b) \quad (1.5.1)$$

Yes, such an operation requires two multiplications instead of one. But if one uses 50 safe operations in the beginning of the calculation, 50 safe operations in the end of the calculation, while a million operations in the middle stay ordinary, then the benefit of using the smaller transform outweighs the increased complexity of a few exceptional operations.

Such an approach may seem risky, but for many algorithms used in practice there exist methods of a relatively fast correctness check. Exponentiation algorithms, which are the foundation of many primality tests and factorization methods, can be checked for correctness of the result by Gerbicz-Li algorithm. Operations on Edwards elliptic curves can be checked by evaluating the curve equation at the calculated coordinates. One can’t avoid roundoff errors by applying such methods, but can detect them in the unlikely case of a pathological operation in the middle of calculation. Using strong correctness checks is recommended anyway, because they also help to detect hardware and software failures.

Also, when implementing transforms over the complex field, it is advised to use methods that decrease the expected value of resulting coefficients. One of such methods is the ‘balancing’: adopting a balanced representation with the norm $-\frac{1}{2}base \leq a_i < \frac{1}{2}base$. It’s quite easy to balance a representation, one just needs to perform the following operation:

$$\text{if } a_i > \frac{1}{2}base \text{ then } a_i \leftarrow a_i - base, a_{i+1} \leftarrow a_{i+1} + 1 \quad (1.5.2)$$

For example, number 17 can be represented as $\{2, -3\}$ in base 10.

On average, the balancing is decreasing the expected absolute value of resulting coefficients by a factor proportional to $\sqrt{N}$. But there are exceptions to this too, as the balancing does not work as good for squaring. One should keep this in mind when selecting N , or alternatively all squarings can be made by using the safe method (1.5.1).

1.6 Finite field

The fast multiplication algorithm described above can be modified in a way that allows the quotient ring modulo a prime number to be selected as the field of coefficients $\mathbb{F} = \mathbb{Z}_p$. Indeed, division can be replaced by multiplication by the inverse, and i replaced by a primitive 4-th root of unity. The only condition is that there should exist a primitive N -th root of unity in $\mathbb{Z}_p$.

The modified algorithm is called ‘FFT over finite field’ or ‘number-theoretic transform’ (NTT), although from the purely algorithmic point of view it is identical to the complex field version. If a programming language supports polymorphism, then a single implementation of fast multiplication can be written. Such an implementation should be able to work both with $\mathbb{C}$ and $\mathbb{Z}_p$, depending on the type of input data.

But, of course, for the optimal performance peculiar properties of modular arithmetic should be taken into account. First of all, resulting coefficients are still limited by the data type. Yes, there are no roundoff errors in $\mathbb{Z}_p$, but there are overflow errors. This happens because the resulting coefficients have two distinct meanings. During multiplication they belong to $\mathbb{Z}_p$, but in the normalization stage they are treated like they belong to $\mathbb{N}$. Of course, the result is always correct in $\mathbb{Z}_p$, but if the modulo operation is applied to even a single coefficient of the result, then the identity $\mathbf{a} \cdot \mathbf{b} = P(base)$ becomes false.

This fact forces to choose a relatively big p as the modulus, which is not convenient. Modern parallel processors (graphical accelerators) are optimized for 32-bit calculations. Either one chooses 32-bit p , which severely limits N and $base$, or one uses much slower 64-bit arithmetic. But this dilemma has an elegant solution. Since the result is always correct in $\mathbb{Z}_p$, one can choose several relatively small p_i , run the multiplication algorithm independently for each modulus, and then reconstruct the coefficients modulo $\prod p_i$ in the normalization stage with the help of the Chinese remainder theorem.

As an example, Genefer utility (<https://github.com/galloty/genefer22>) chooses 32-bit prime numbers $125 \cdot 2^{25} + 1$, $243 \cdot 2^{24} + 1$, $235 \cdot 2^{24} + 1$. These prime numbers allow to perform multiplication modulo $x^N - 1$ with transform size 2^{24} and multiplication modulo $x^N + 1$ with transform size 2^{23} . The reconstructed coefficients can store up to 95 bits of data, allowing to choose a very big representation base.

1.7 Hierarchical memory

Modern computing devices employ a rather complicated system of data storage. As a rule, the exact topology of the system is hidden from the program being executed, but nonetheless it has a significant influence on the performance of calculations. Each core of a typical modern multi-core central processor has a small first-level cache for data and instructions and a somewhat larger second-level cache for data. In addition to that, the processor has a common third-level cache, reaching megabytes and tens of megabytes in size, which is accessible by all cores equally. In a sense, the third-level cache is a local memory of the processor. Data from

the slow main memory located outside the processor is copied into the local memory for processing. Main memory in multiprocessor systems can also be subdivided into ‘local’ (physically connected to the processor) and ‘foreign’ (connected to another processor of the system). All this complicated topology is hidden behind the flat address space allocated by operating system to the executed program.

However, high-performance computing has to take into account the hierarchical organization of memory on all levels. One should make sure that all data is stored only in local main memory, and try to avoid situations when the data necessary to perform a single operation is larger than the third-level cache. From the point of view of big number multiplication it means that all data being transformed should fit the third-level cache, because each step of the recursive algorithm uses all values. It is also possible in multi-core processors to have a situation when each core works on its own transform which fits the common cache, but transforms of all cores do not fit together. The solution to this is to have several cores work on the same transform in parallel. A possible inefficiency caused by the synchronization overhead should be more than compensated for by avoiding the need to access the slow main memory.

Some optimization is also possible even at the lower levels. Each individual core may benefit from having all the necessary data in its own second-level cache. From its point of view the third-level cache is a “slow” memory too. However, the second-level cache is not big enough to fit transforms of any significant size. A method called ‘multi-pass transform’ [1] was developed for situations exactly like that.

The method is based on the convenient properties of the roots of polynomial $x^N - 1$. If a transform is applied several times to the same data, the result is still composed of values of polynomial $A(x)$ at some roots of polynomial $x^N - 1$. One can get the correct transformation by performing several passes of a shorter transform, if the passes are set up in a proper way and the result is additionally multiplied by missing roots.

As a first step, transform size N is split into two close values $N = N_1 \cdot N_2$. After that the input data is considered as a $N_1 \times N_2$ matrix, and the following steps are performed:

1. Perform N_1 transforms of size N_2 .
2. Transpose the data into the $N_2 \times N_1$ matrix.
3. Multiply the data by corresponding roots, which can be calculated as $i^{jk/N}$.
4. Perform N_2 transforms of size N_1 .

On each step one reads a fixed amount of data from the slow memory to the local memory, performs the necessary operations and writes the data back. It’s easy to come up with a few optimizations that decrease the number of accesses to the slow memory. The third step can be merged with the fourth one. The first two steps can be merged into the following repeating sequence:

- Copy a column of height N_2 into the local data row.
- Perform the transform of size N_2 .
- Write the local data row into the column of height N_2 .

As a result of the optimizations, the transform is performed with 2 reads from the slow memory and 2 writes to the slow memory.

The inverse transform can be implemented with a symmetrical sequence of operations. But an additional optimization is possible for the inverse transform, which can significantly improve performance. In practice, the result of big number multiplication is often used only as an input to the next multiplication operation. In other words, after the normalization stage ends the next transform starts. If data is arranged in a proper way, the ends of multiplication operations can be fused:

- The last step of the inverse transform fills the local data row with the resulting coefficients.

- Perform the normalization.
- Perform the first step of the forward transform.
- Write the local data row into the slow memory.

The result becomes partially transformed, which allows to save 1 read and 1 write to the slow memory. This optimization may lead to up to 20% in performance gain on long chains of multiplications or squarings, which are often observed in primality tests. All this thanks to taking into account the specifics of hierarchical memory organization.

Similar optimizations are possible on the level of operations on big numbers. Consider a pair of expressions that need to be evaluated $(\mathbf{a} + \mathbf{b}) \cdot \mathbf{c}$ and $(\mathbf{a} + \mathbf{b}) \cdot \mathbf{d}$. It's obvious that to minimize the number of operations one needs to compute and store the intermediate value $\mathbf{a} + \mathbf{b}$ first. If done this way, 6 reads and 3 writes are necessary. Independent evaluation of expressions leads to 6 reads and 2 writes. When dealing with really big numbers, memory access becomes the bottleneck, and one may actually increase performance by decreasing the number of memory accesses despite the increase in the number of operations.

1.8 Modular multiplication

When solving problems in computational number theory, most of the time one needs to compute not the product of two big numbers, but the product modulo a fixed big number. An additional modulo operation needs to be performed on the product, which generally requires performing of the division operation. Fortunately, the modulus is fixed and is not changing between operations, which allows to replace the expensive division operation with two multiplications. In order to do so, one needs to compute polynomial $D(x)$ called ‘pseudoreciprocal’ of modulus $M(x)$:

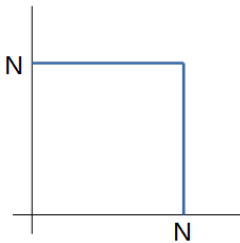
$$D(x) \times M(x) = x^N \quad (1.8.1)$$

It can be done relatively fast with the help of Newton’s method [2]. Note that N should be at least two times larger than the degree of $M(x)$.

The next step is to compute polynomial $T(x) = A(x) \times B(x)$, the result of multiplication. If each multiplier is less than the modulus, then $\deg(T) \leq 2\deg(M) < N$. Now one is ready to compute the quotient $Q(x)$ and the remainder $R(x)$ such that $T(x) = Q(x) \times M(x) + R(x)$. Using (1.8.1)

$$Q(x) = \frac{T(x) \times D(x)}{x^N}, \quad R(x) = T(x) - Q(x) \times M(x) \quad (1.8.2)$$

Division by x^N is implemented by discarding all coefficients with indexes smaller than N and decreasing indexes of the rest of coefficients by N .



Let’s look at the $x^N - 1$ fast multiplication algorithm. In case the sum of multiplier degrees is exceeding N , the result is computed modulo $x^N - 1$:

$$P^{mod}(x) = P(x) \bmod (x^N - 1) \quad (1.8.3)$$

It’s easy to get the relation between the coefficients of $P(x)$ and $P^{mod}(x)$, one just needs to perform the long division:

$$mod_p_k = p_k + p_{N+k} \quad (1.8.4)$$

Figure 1.5: Number of terms in mod_p_k .

Essentially, the part that exceeds N is cyclically wrapped and added to the lower terms. A similar relation holds for the multiplication modulo $x^N + 1$, the only difference is that the addition in (1.8.4) is replaced with subtraction, which explains why the operation is sometimes called “negacyclic”.

To perform the multiplication $T(x) \times D(x)$ in (1.8.2), one needs a transform of size $N + \deg(M)$ at least. But note that most of the result is not used at all, N lowest coefficients are just discarded, therefore there's no need to compute them at all. If one zeroes out the first $\deg(M)$ coefficients of $T(x)$, essentially just setting $p_k = 0$ for the lower terms, then the only part of the product which is needed for the following operations becomes wrapped and added to the zeroes in the process of multiplication modulo $x^N - 1$:

$$Q(x) = (T(x) - T(x) \bmod x^{\deg(M)}) \times D(x) \bmod (x^N - 1) \bmod x^{\deg(M)} \quad (1.8.5)$$

Thanks to this trick, both multiplications in (1.8.2) can be performed with the same transform size N which is used for multiplication of the input numbers.

Thus, modular multiplication of big numbers turns out to be three times slower than just multiplication of big numbers. This is a very distressing fact. But often when solving problems in computational number theory one encounters moduli which are not just some random big numbers, but numbers defined with simple algebraic forms. For some of the forms more efficient methods to compute the remainder are known.

Once again, the fast multiplication algorithm computes the result modulo $x^N - 1$ or $x^N + 1$. Substituting the base of representation for x , we have:

$$\mathbf{a} \cdot \mathbf{b} \bmod (\text{base}^N \pm 1) = [P(x) \bmod (x^N \pm 1)](\text{base}) \quad (1.8.6)$$

In other words, for moduli of $b^n \pm 1$ form one can choose $\text{base} = b$, $N = n$ and *do nothing else*. The remainder is computed automatically as the result of the fast multiplication.

Fermat numbers $2^{2^n} + 1$ and generalized Fermat numbers $b^{2^n} + 1$ have such form. N equals a power of two for them, which allows to use the most efficient implementations of the multiplication modulo $x^N + 1$. A downside of this approach is that N is fixed, which means that the limits imposed on base by the underlying data type become the limits on b . For finite fields this problem can be solved by employing the Chinese remainder theorem, but for the complex field more complicated solutions are needed. One of them is the choice of $\text{base} = \sqrt{b}$, $N = 2n$, although the irrational base requires more accurate handling of the precision of representations.

Generalized Fermat numbers (GFN) are very popular among enthusiasts who search for large prime numbers. They are present in great quantity in the list of the largest known prime numbers. In no small part thanks to the speed of the modular multiplication.

1.9 Weighted transform

Another popular class of numbers is Mersenne numbers. Most of the time since the beginning of the Renaissance the largest known prime number was a Mersenne number. They have $2^n - 1$ form, but only $2^p - 1$ can be prime, where p is a prime number itself. Using a prime number as the transform size is possible but not recommended because it is the worst case from the performance point of view. In 1994, Crandall and Fagin introduced a multiplication method specifically targeting Mersenne numbers, which they called 'irrational base discrete weighted transform' (IBDWT) [3]. They suggest to use $\text{base} = 2^{n/N}$, $N < n$ for some convenient N of our choice, like a power of two.

Of course, the question how can one construct such a representation arises immediately. The most obvious way is to construct the representation as digits in base $2^{n/N}$ numeral system, although one may have trouble maintaining the necessary precision. Crandall and Fagin suggest a method of 'weighing'.

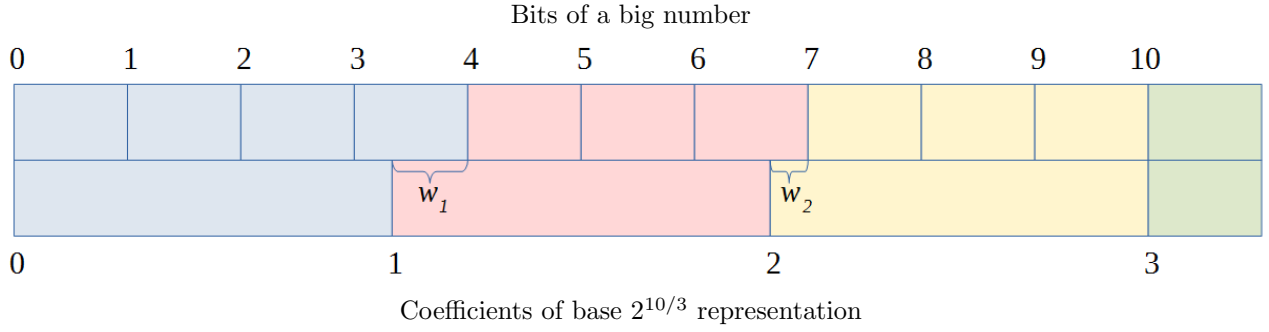


Figure 1.6: Big number in the binary form and its irrational base representation.

Suppose we've chosen $base = 2^{10/3}$ (fig. 1.6). Then the first few terms of the representation are:

$$A(base) = a_0 + a_1 \cdot 2^{10/3} + a_2 \cdot 2^{20/3} + \dots$$

while the binary form of the big number looks like this:

$$\begin{aligned} \mathbf{a} &= \mathbf{a}_{[0123]} + \mathbf{a}_{[456]} \cdot 2^4 + \mathbf{a}_{[789]} \cdot 2^7 + \dots \\ &= \mathbf{a}_{[0123]} + \mathbf{a}_{[456]} \cdot 2^{4-10/3} \cdot 2^{10/3} + \mathbf{a}_{[789]} \cdot 2^{7-20/3} \cdot 2^{20/3} + \dots \end{aligned}$$

By combining, we can construct the representation using bit substrings of the big number and irrational powers of two:

$$a_0 = \mathbf{a}_{[0123]}, \quad a_1 = \mathbf{a}_{[456]} \cdot 2^{4-10/3}, \quad a_2 = \mathbf{a}_{[789]} \cdot 2^{7-20/3}, \dots$$

Yes, the coefficients of the representation become larger than the base. But there's no problem in that, the only thing that matters is $A(base) = \mathbf{a}$. Besides, the overhead is always less than 2 times (less than 1 bit). The irrational powers of two, which are multiplied by the bit substrings, are called 'weights' and are generally calculated as

$$w_i = 2^{\lceil in/N \rceil - in/N} \in [1, 2) \quad (1.9.1)$$

As a result, we have the following representation

$$a_i = w_i \mathbf{a}_{[\lceil in/N \rceil, \lceil (i+1)n/N \rceil)} \quad (1.9.2)$$

Easy to see that the length of a bit substring is either $\lfloor n/N \rfloor$ or $\lceil n/N \rceil$ (3 and 4 in the example above). And they can be balanced in the same way as in (1.5.2).

The weighted representation has a property that all $\frac{a_i}{w_i}$ are integers. But what can be said about the product of two such polynomials? The grade-school multiplication method tells us that each p_k is a sum of products $a_i b_j$ where $i + j = k$. But then $\frac{p_k}{w_k}$ can be seen as a sum of integers (bit substrings) multiplied by the following weights:

$$\begin{aligned} \frac{w_i w_j}{w_{i+j}} &= 2^{\lceil in/N \rceil - in/N + \lceil jn/N \rceil - jn/N - \lceil (i+j)n/N \rceil + (i+j)n/N} \\ &= 2^{\lceil in/N \rceil + \lceil jn/N \rceil - \lceil in/N + jn/N \rceil} \end{aligned} \quad (1.9.3)$$

A sum of rounded up values is either equal to or exceeds by one the rounded up sum of values. Therefore (1.9.3) is equal either to 1 or to 2. $\frac{p_k}{w_k}$ is always an integer. This stays true even in case of multiplication modulo $x^N \pm 1$. This follows from (1.8.4) and the fact that $w_i = w_{N+i}$.

Using (1.2.2) and (1.9.1), we can write the result of multiplication in the following way:

$$\mathbf{a} \cdot \mathbf{b} = P(2^{n/N}) = \sum p_k 2^{kn/N} = \sum \frac{p_k}{w_k} 2^{\lceil kn/N \rceil} \quad (1.9.4)$$

p_k and $2^{kn/N}$ are irrational numbers, but $\frac{p_k}{w_k}$ and $2^{\lceil kn/N \rceil}$ are integers, because all of the irrationality cancels itself. All one needs to do to obtain integers is to round the values (and calculate the roundoff error for quality control of the choice of N). It means that the normalization stage can deal only with integers.

Once again, the weighted transform described above can be used to automatically reduce the result modulo $base^N \pm 1 = 2^n \pm 1$. But a similar transform can be constructed for moduli of some other forms too, like Proth numbers $k \cdot 2^n + 1$ and numbers of $k \cdot 2^n - 1$ form, which are sometimes erroneously called Riesel numbers. For such moduli, the representation uses $base = 2^{(n+\log_2 k)/N}$. The weights are calculated as

$$w_i = 2^{\lceil i(n+\log_2 k)/N \rceil - i(n+\log_2 k)/N} \in [1, 2) \quad (1.9.5)$$

Unfortunately, $w_i \neq w_{N+i}$:

$$\begin{aligned} \frac{w_{N+i}}{w_i} &= 2^{\lceil n+\log_2 k + i(n+\log_2 k)/N \rceil - n - \log_2 k - i(n+\log_2 k)/N - \lceil i(n+\log_2 k)/N \rceil + i(n+\log_2 k)/N} \\ &= \frac{1}{k} 2^{\lceil \log_2 k + i(n+\log_2 k)/N \rceil - \lceil i(n+\log_2 k)/N \rceil} \end{aligned} \quad (1.9.6)$$

It means that $\frac{p_t}{w_t}$ has the factor $\frac{1}{k}$ and is not integer anymore. This fact significantly complicates the normalization stage. There's a way to deal with the complication, the main idea of which is to premultiply all big numbers by k^{-1} . In order for the result of big number multiplication to have the same factor, it needs to be additionally multiplied by k in the normalization stage.

$$(a \cdot b)k^{-1} = k(a k^{-1} \cdot b k^{-1}) = k \sum p_t 2^{t(n+\log_2 k)/N} = \sum \frac{k p_t}{w_t} 2^{\lceil t(n+\log_2 k)/N \rceil} \quad (1.9.7)$$

As a result, $\frac{k p_t}{w_t}$ produces an integer value, and the normalization stage can stay integer.

The weighted transform can also be used for numbers of $k \cdot b^n \pm 1$ form. But $base = 2^{(n \log_2 b + \log_2 k)/N}$ can't be used anymore, because $\frac{w_{N+i}}{w_i}$ turns out to be irrational, that's why $base = b^{(n+\log_b k)/N}$ is used. It means that to construct the representation one uses not the bit substrings, but substrings of a representation with base b . The weights are calculated similar to (1.9.5), only the base of exponentiation is replaced with b :

$$w_i = b^{\lceil i(n+\log_b k)/N \rceil - i(n+\log_b k)/N} \in [1, b) \quad (1.9.8)$$

For numbers of $b^n \pm c^m$ form the multiplication is performed modulo $b^n c^{-m} \pm 1$ [4]. The weights take into account both bases:

$$w_i = b^{\lceil in/N \rceil - in/N} c^{\lceil -im/N \rceil + im/N} \in [1, bc) \quad (1.9.9)$$

The resulting weights belong to a larger interval which also depends on c . But luckily $w_i = w_{N+i}$ and $\frac{w_i w_j}{w_{i+j-N}} = c$ in the most popular case $m = 1$ (which is equivalent to $\text{mod } p_k = p_k \mp c p_{n+k}$, which is equivalent to calculating the remainder of long division by $b^n \pm c$). The normalization stage deals only with integers, and there's no need to take any additional action to support $c \neq 1$.

The weighted transform is most easily implemented over the complex field. But when looking at expressions where even the exponents are irrational, one may ask a question of how much precision is necessary to obtain the correct result? Indeed, even when using the double-precision floating-point data type the weights are calculated with a significant numerical error, which keeps accumulating in the following calculations. But we do not need to do the hard work of estimating the error because we know exactly what it is. The numerical error equals to the fractional part of a value that should be integer. Only thanks to the ability to cancel the accumulated error by a simple rounding operation, the weighted transform has become so successful in practice.

An implementation over $\mathbb{Z}_p$ is possible too, but requires quite large prime modulus which does not fit into 32 bits. Even 64-bit arithmetic allows to implement only the simplest forms of weighing, like $k = 1$.

GWNum library (<https://www.mersenne.org/download/>) implements the weighted transform over the complex field for numbers of $k \cdot b^n + c$ form for a wide range of parameters.

1.10 Free word space

The fastest and the most convenient choice of the transform size N is a power of two, transform sizes with small prime factors like 3 or 5 are also used in practice. Their benefit is providing a smoother transition between transforms, making the next transform less than 2 times larger than the previous one. Nonetheless, there is not enough smoothness, transforms grow in large steps, after the switch to the next one roundoff errors drop to levels much lower than the threshold. The difference between the current level of roundoff errors and the threshold is called ‘free word space’. It is the area above the shaded one and below S on fig. 1.4. The difference can be seen as some kind of unused space left in each coefficient (word) of the result before the normalization. And we can make use of this space.

When we were reconstructing the coefficients of the result in section 1.3, we never used the fact that the result is a product of two polynomials (1.2.1). The only things that matter to the inverse transform are the degree of the resulting polynomial and the maximum size of the coefficients. There are several operations beside $\mathbf{a} \cdot \mathbf{b}$ that produce polynomials of the same degree with a slightly larger coefficients. For example, $\mathbf{a} \cdot \mathbf{b} + \mathbf{c}$ or $(\mathbf{a} + \mathbf{b}) \cdot \mathbf{c}$ or even $\mathbf{a} + \mathbf{b}$. In other words, we can calculate an entire expression (of the second degree at most) and obtain the result by performing the inverse transform only once.

Let’s define the following functions for transforming numbers:

transform Performs a forward transform $\bar{A} = \text{transform}(A)$.

inv_transform Performs an inverse transform $A = \text{inv_transform}(\bar{A})$.

carry Performs normalization and modulo operation (if necessary) $A = \text{carry}(A)$.

Arguments of the functions are vectors which contain either polynomial $A(x)$, or its values at the appropriate roots: $\bar{A} = \{A(r_i)\}$. All arithmetic operations on vectors are performed per-element.

We can express multiplication of two big numbers $\mathbf{a} \cdot \mathbf{b}$ in the following notation:

$$\text{carry}(\text{inv_transform}(\text{transform}(A) \cdot \text{transform}(B))) \quad (1.10.1)$$

Expression $\mathbf{a} \cdot \mathbf{b} + \mathbf{c} \cdot \mathbf{d}$ can be evaluated in the following two ways, using already transformed data:

$$\text{carry}(\text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B})) + \text{carry}(\text{inv_transform}(\bar{C} \cdot \bar{D}))) \quad (1.10.2)$$

$$\text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B} + \bar{C} \cdot \bar{D})) \quad (1.10.3)$$

The second way requires only one inverse transform instead of two, and only one normalization instead of three.

It is important to remember, though, that each arithmetic operation performed on transformed data increases the maximum size of coefficients (words), because normalization is not possible before the inverse transform is performed.

$$\bar{A} + \bar{B} \neq \text{transform}(\text{carry}(\text{inv_transform}(\bar{A} + \bar{B}))) \quad (1.10.4)$$

It means additional terms require an additional space in words. How much free space is needed is hard to estimate theoretically, but easy to calculate in practice by evaluating various expressions with random inputs and keeping track of roundoff errors. GWNuM library offers programming interface which helps users to answer questions about whether an expression is safe to be calculated using the given N and the given big number.

In the process of gathering roundoff error statistics of various expressions, several relationships were discovered. For example, $\mathbf{a}^2$ and $(\mathbf{a} + \mathbf{b}) \cdot \mathbf{c}$ require about the same free word space. It means that if a transform can be used to perform squaring of a big number, then it can be used to calculate $(\mathbf{a} + \mathbf{b}) \cdot \mathbf{c}$ too.

Several similar classes of expressions are listed in the following table, sorted by the amount of free word space needed:

$a^2, (a+b) \cdot c, a \cdot b + c, a \cdot b + c \cdot d$
$(a+b+c) \cdot d, a^2 + b \cdot c, (a+b) \cdot c + d \cdot e$
$a^2 + b^2, (a+b) \cdot (c+d), (a+b+c+d) \cdot e, (a+b) \cdot c + (d+e) \cdot f$
$a^2 + a^2, (a+b)^2, (a+b+c+d+e+f+g+h) \cdot i, (a+b+c+d) \cdot (e+f),$ $(a+b) \cdot (c+d) + (e+f) \cdot (g+h), (a+b+c+d) \cdot e + (f+g+h+i) \cdot j$

Ability or inability to use those expressions for a particular big number leads to the situation when some algorithms can have several implementations of choice, depending on the available free word space. The fastest implementation requires the largest amount of free word space. If it is not available, one can either increase N , which will significantly slow down all operations, or use another implementation only a bit slower. For example, the algorithm of doubling a point on an Edwards elliptic curve can have four implementations [5].

There's another intriguing possibility. If one doubles N , then expressions of the 4-th degree become safe to calculate. The mentioned algorithm of doubling a point on an Edwards elliptic curve has the degree of all formulas equal to four. Unfortunately, it also has many squarings and 4-th powers, which raise roundoff errors to the level when even doubling of the transform size N is not enough for most big numbers. But when it's enough, calculating formulas with a single inverse transform gives 5% performance gain over the regular implementation.

Here and later algorithms and operations performed on transformed big numbers are called 'nested', because they can be thought of as executing inside a transform. A useful property of nested algorithms is that they are independent of the transform type and modulo operation. If the modulo operation exists (either as a separate operation or as a weighted transform), one gets the reduced output of the entire algorithm immediately, without the need to apply modulus to each term of the expression individually.

1.11 Nested multiplication

Among all possible nested algorithms, nested multiplication of polynomials is of particular interest. Indeed, multiplication of polynomials is widely used in various methods of integer factorization, and of big number factorization too. Usually, such applications require computing the resulting coefficients modulo some big number. Let $A(y)$ and $B(y)$ be two polynomials with big coefficients a_i and b_j , and let m be a big modulus. We can compute the product $P(y)$ of degree $M-1$ by using the grade-school multiplication:

$$P(y) = A(y) \times B(y) \quad (1.11.1)$$

$$p_k = \sum_{i+j=k} a_i \cdot b_j \bmod m \quad (1.11.2)$$

To find the resulting coefficients, one may execute the fast multiplication algorithm $a_i \cdot b_j \bmod m$ on each pair of coefficients independently, and then sum up all results. But (1.11.2) is an expression of the second degree, which means we can compute it using only one inverse transform. Of course, we'll need an additional free word space, but not much of it. Note that when M is increasing, only the quantity of terms is increasing in the resulting sum (fig. 1.1), the size of the terms stays the same. This means that the amount of required free word space depends on $\log_2 M$. In practice, a transform size one step above the minimal has enough free word space to perform multiplication of even the largest polynomials. We call the algorithm which performs multiplication this way a 'nested grade-school multiplication'.

Let's look closer at how the algorithm works. As a first step, all big numbers need to be transformed. Let's designate the transformed big coefficients as $\bar{a}_i$ and $\bar{b}_j$. Each of them consists of N elements, which

can be designated as $\bar{a}_{i\ell}$ and $\bar{b}_{j\ell}$. Next, $\bar{p}_{k\ell}$ is computed for each ℓ using the following formula:

$$\bar{p}_{k\ell} = \sum_{i+j=k} \bar{a}_{i\ell} \cdot \bar{b}_{j\ell} \quad (1.11.3)$$

In the context of the operation ℓ is a constant. This allows us to redefine the formula in terms of polynomials:

$$P_\ell(y) = A_\ell(y) \times B_\ell(y) \quad (1.11.4)$$

Note that at this moment we do not consider the data to be transformed anymore. Now it is just two polynomials for us to multiply. The product $P_\ell(y)$ is a degree $M - 1$ polynomial. To compute it, we can use the fast multiplication algorithm modulo $y^M - 1$ described above, but without the normalization stage. For each ℓ , the transform of size M is used to obtain $\bar{A}_\ell$ and $\bar{B}_\ell$, then $\bar{P}_\ell$ is computed, and the inverse transform of size M is applied to it. In the end, the inverse transform of size N is applied to each of $\bar{p}_k$. We call the resulting algorithm a ‘nested fast multiplication’ or ‘nested FFT’ or even ‘FFT inside FFT’.

The important difference between nested fast multiplication and multi-pass fast multiplication is the fact that the nested transform and the outer transform are completely independent of each other. The outer transform can be a weighted transform modulo $x^N + 1$, while the nested transform is a transform modulo $y^M - 1$. Completely different sets of $\{r_i\}$. Furthermore, in an implementation of nested multiplication the choice of transform depends only on M , and is completely independent of the outer transform. The only thing that matters is that the nested transform have to be executed N times.

As in any nested algorithm, the outer modulo operation is applied only once for each resulting coefficient, even if it is a sum of millions of terms. This leads to a situation when for small M and large N the execution time is dominated by the outer transform (and modulo operation, if it is explicit), the complexity of the algorithm is close to $O(M)$. This is especially true when the transform of size M fits into the second-level cache, while the transform of size N requires the multi-pass algorithm and access to slow memory. In other words, multiplication of polynomials can be performed in linear time in certain circumstances (fig. 1.7).

On the other hand, for large M the nested multiplication exhibits another useful property. It requires only a single transform of size M to be stored in memory, data for each subsequent ℓ (or a “line”) can be written into the same memory block the previous line used. Also, there’s no need to keep zero-padding coefficients if the degree of $\mathbf{A}(\mathbf{y})$ or $\mathbf{B}(\mathbf{y})$ is less

than M , zero-padding can be introduced into the each individual line right before the transformation. Furthermore, the coefficients of polynomials $\mathbf{A}(\mathbf{y})$ and $\mathbf{B}(\mathbf{y})$ can be reused to store polynomial $\mathbf{P}(\mathbf{y})$, because after a line is copied, the values at the original memory regions are never accessed again. All of this allows to perform multiplication of very large polynomials using relatively small amount of memory. In practice it makes possible to multiply two million-degree polynomials, each coefficient of which contains more than a thousand digits, using only a personal computer, with the operation taking just a few seconds.

Performing the nested multiplication modulo $y^M - 1$ is equivalent to applying modulus $\mathbf{y}^M - 1$ to polynomial $\mathbf{P}(\mathbf{y})$. This allows to perform operations modulo a big polynomial in the same way as described in section 1.8. Besides, in a situation when one needs only a subset of the resulting coefficients, it is possible to save time by not performing the inverse transform for the unneeded coefficients.

Another possible optimization is to keep the transformed data of each line of a particular polynomial and reuse it later. For example, if polynomial $\mathbf{B}(\mathbf{y})$ is used unchanged as an input to several multiplication

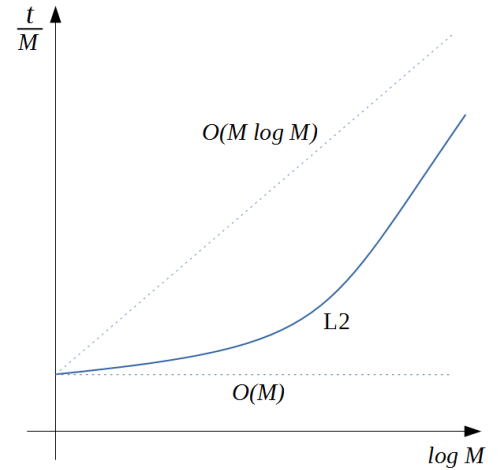


Figure 1.7: Performance of nested multiplication of degree- M polynomials before and after exceeding limits of second-level cache L2.

operations, then one can keep $\{\overline{B}_\ell\}$ instead of the polynomial. However, this approach requires more memory if $\deg(\mathbf{B}) < M$.

Polymult package of GWNu library (<https://www.mersenne.org/download/>) implements nested multiplication modulo any big number supported by the library.

1.12 Implementation

As already noted, a software implementation of the described algorithms heavily depends on the architecture of a particular computing device. To reach the maximum performance one needs to take into account hierarchical memory organization and availability of vector/parallel processor instructions. Besides that, an important factor is the choice of programming language.

High-level programming languages rarely provide access to vector processor instructions, and in case they do, performance of the resulting executable may vary depending on version and release date of the compiler. This happens because a sequence of operations often can be translated into processor instructions in several ways. For example, an operand residing in the main memory can be used directly, or be first loaded into a processor register. Which way is the fastest can depend on the architecture of a particular processor model or processor family. Furthermore, the optimization strategy employed by a compiler can depend on many things, including which classes of tasks are most popular among developers at the time of the compiler release. This leads to a situation when performance of an implementation written in a high-level language varies from year to year in both ways.

Assembler language, on the other hand, has its own set of problems. First of all, one needs to have a separate implementation for each instruction set, for each processor architecture, for each type of transforms. Any possibility of porting of the implementation to other computing devices is lost. The source code becomes extremely complicated and demands the highest level of talent to support. But in the end, the implementation can reach the highest performance possible on a given hardware.

As a compromise, one may implement transform and normalization modules in assembler, while providing access to the modules from a higher level language. The next level of abstraction should manage hardware-dependent implementations and hide all specific calls behind a single programming interface of elementary operations.

From the point of view of memory access, the most efficient way to perform a transform is to do it in-place, overwriting existing data in caches of all levels. The size of transformed and untransformed data is the same for the finite field transforms (if zero-padding is accounted for). But for transforms over the complex field, the untransformed data has real values, while the transformed data has complex values, which require twice as many floating-points values to store. Luckily, the transforms discussed in this work have a property called Hermitian symmetry, which means that symmetrical complex values differ only in the sign of the imaginary part. This allows to halve the amount of memory required. Transformed data of multiplication modulo $x^N - 1$ consists of two real values and $N/2 - 1$ pairs of complex values, transformed data of multiplication modulo $x^N + 1$ consists of $N/2$ pairs of complex values. Both kinds of transformed data can fit into the same memory block allocated for the untransformed N coefficients.

The programming interface should pay a lot of attention to tracking the actual state of all numbers used in an operation. A big number can be stored either in an untransformed state, or in a transformed state, or in a partially transformed state as described in section 1.7. Performing of an elementary operation may change the state not only of the result, but of all other operands too. For example, when performing multiplication operation $A = A \times B$, the number A is overwritten with the untransformed or partially transformed result, while the number B becomes the transformed $\overline{B}$. In case users want to keep B untransformed, they should either make a copy, or somehow communicate their desire to the programming interface, so that it can make use of a temporary number to hold $\overline{B}$. The same principle applies to performing the partial transformation

during normalization, users should indicate if it is desired or not. The programming interface can't deduce it, because it depends on the way the number is used in the following operations. If it is used in a multiplication, then partial transformation increases performance. But some operations require untransformed inputs, like the operation of converting into the binary form or the addition with normalization. If a number is in a transformed state, then the programming interface has to perform the inverse transform first (which is sometimes easier to do by multiplication by 1).

As mentioned, the addition with normalization requires untransformed numbers as its input. One can perform addition of transformed numbers too, but the result of the operation will have a slightly different meaning. Normalization of such a result is not possible, and it can be used safely as an input to the multiplication operation only if there's enough free word space available, as discussed in section 1.10. The programming interface should track the number of unnormalized additions/subtractions and provide instruments to estimate the safety of subsequent operations.

Besides addition and subtraction, the programming interface may offer the combined operation $A' = A+B$, $B' = A - B$. This operation is trivial to implement per-element and does not require storing of temporaries. Some algorithms, like described in [5], can benefit from using such an operation, especially in unnormalized form.

The following table lists elementary operations recommended to be implemented in a programming interface. `carry()` function besides normalization and modulo operations can also optionally perform multiplication by small constant, addition of small constant and partial forward transform.

$X \leftarrow A + B$	$\bar{X} \leftarrow \text{transform}(A)$
$X \leftarrow A - B$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A}))$
$X, Y \leftarrow A \pm B$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B}))$
$X \leftarrow \text{carry}(A + B)$	$X \leftarrow \text{carry}(\text{inv_transform}((\bar{A} + \bar{B}) \cdot \bar{C}))$
$X \leftarrow \text{carry}(A - B)$	$X \leftarrow \text{carry}(\text{inv_transform}((\bar{A} - \bar{B}) \cdot \bar{C}))$
$X, Y \leftarrow \text{carry}(A \pm B)$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B} + \bar{C}))$
$\bar{X} \leftarrow \bar{A} + \bar{B}$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B} - \bar{C}))$
$\bar{X} \leftarrow \bar{A} - \bar{B}$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B} + \bar{C} \cdot \bar{D}))$
$\bar{X}, \bar{Y} \leftarrow \bar{A} \pm \bar{B}$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B} - \bar{C} \cdot \bar{D}))$

Number X can be the same as any of A, B, C, D . This may affect the implementation of an operation.

Chapter 2

Exponentiation

The operation of raising a big number a to a big power e modulo a big number m is one of the most popular tasks performed with big numbers. Many primality tests use exponentiation one way or another. One can find many exponentiation algorithms described in literature [6], in this chapter we have a closer look at some of them being applied to big numbers.

2.1 Binary exponent

Raising a number to a large power e with millions of decimal digits may appear daunting initially. However, it can be computed quite efficiently when all operations are performed modulo some number. This ensures that the result does not grow exponentially and remains within the bounds of the modulus. Another valuable technique is squaring, which involves multiplying a value by itself and effectively doubles the exponent when applied iteratively. By repeatedly applying squaring, one can attain an exponent comparable to e in just $\log_2 e$ steps.

The number e is represented in binary form as

$$e = e_0 + e_1 2 + e_2 2^2 + e_3 2^3 + \dots \quad (2.1.1)$$

Bits where $e_i = 0$ have no effect on the result of exponentiation. Therefore, we can express this as

$$\begin{aligned} a^e \bmod m &= a^{e_0} \cdot a^{e_1 2} \cdot a^{e_2 2^2} \cdot a^{e_3 2^3} \cdot \dots \bmod m \\ &= \prod_{e_i \neq 0} a^{2^i} \bmod m \end{aligned} \quad (2.1.2)$$

The result is a product of terms, where the terms are powers of the base a raised to the powers of 2 corresponding to the non-zero bits. In other words, one can achieve this by iteratively squaring the base and multiplying the accumulating product by the current value, only if the i -th bit of number e is not zero.

Algorithm 2.1: Right-to-left exponentiation.

Output: $r = a^e \bmod m$

```
1   $r \leftarrow 1, \quad s \leftarrow a$ 
2  for  $i \leftarrow 0$  to  $\lfloor \log_2 e \rfloor$  do
3      if  $e_i \neq 0$  then  $r \leftarrow r \cdot s \bmod m$ 
4       $s \leftarrow s^2 \bmod m$ 
5  end
```

This algorithm is known as ‘right-to-left exponentiation’ because it assumes that the least significant bit of the number e is on the right when represented in textual form, while the most significant bit is on the left.

There is an opposite algorithm called ‘left-to-right exponentiation’. As the name suggests, the product is accumulated starting from the most significant bit e_j , with j decreasing to zero. At each step the product is squared, doubling the effective exponent. It means that the value at step j will be squared j times by the end of the algorithm. Therefore, if the j -th bit of number e is not zero, the accumulating product is multiplied by a instead of using the full form of the current term $a^{2^j} \bmod m$. Thus, the value of the accumulating product at each step of the algorithm is equal to $a^{\lfloor e/2^j \rfloor} \bmod m$.

Algorithm 2.2: Left-to-right exponentiation.

Output: $r = a^e \bmod m$

```

1   $r \leftarrow 1$ 
2  for  $j \leftarrow \lfloor \log_2 e \rfloor$  to 0 do
3       $r \leftarrow r^2 \bmod m$ 
4      if  $e_j \neq 0$  then  $r \leftarrow r \cdot a \bmod m$ 
5  end

```

Let’s take an example with exponent $314 = 100111010_2$.

Algorithm 2.1									
i	0	1	2	3	4	5	6	7	8
e_i	0	1	0	1	1	1	0	0	1
s	a	a^2	a^4	a^8	a^{16}	a^{32}	a^{64}	a^{128}	a^{256}
r	1	a^2	a^2	a^{10}	a^{26}	a^{58}	a^{58}	a^{58}	a^{314}

Algorithm 2.2									
j	8	7	6	5	4	3	2	1	0
e_j	1	0	0	1	1	1	0	1	0
r	a	a^2	a^4	a^9	a^{19}	a^{39}	a^{78}	a^{157}	a^{314}

The same number of operations is performed in each of the algorithms: $\lfloor \log_2 e \rfloor + 1$ squarings and a number of multiplications equal to the number of non-zero e_i (Hamming weight). If the exponent is a power of two, then the algorithms are identical. For an arbitrary exponent Hamming weight is $\frac{1}{2} \log_2 e$ on average.

In practical applications of the algorithms, it is often the case that e and m are big numbers, while a is a small constant. For example, 2 and 3 are often chosen as a in Fermat primality test. In such a case, the multiplication by a in algorithm 2.2 is a multiplication by small constant, which can be performed in the normalization stage practically for free. On the contrary, the multiplication in algorithm 2.1 is always a multiplication of two big numbers. As a result, for arbitrary exponents algorithm 2.1 performs 50% slower than algorithm 2.2, even though the number of operations is the same from the mathematical point of view. Such effects arise from the peculiarities of operations on big numbers, and are the main focus of this chapter.

2.2 Gerbicz check

In addition to performance, reliability is also a crucial aspect of any computation. An error in just one bit can result in catastrophic consequences. One of the most prevalent applications of big numbers involves searching for prime numbers with millions of decimal digits. Due to their size, testing the primality of a single candidate can take several days. Any miscalculation during this process can yield an incorrect outcome for the entire test, potentially misidentifying a prime number as composite. Certain mathematical conjectures, such as the Sierpiński problem, rely on identifying specific prime numbers. These conjectures often define numerical sequences of candidates with numbers growing at an exponential rate. If, by chance, the first prime number in such a sequence is missed today, it could lead to centuries of unnecessary efforts in the search for the subsequent prime number. The magnitude of wasted energy due to a single-bit error can be enormous.

The situation can become even more precarious in volunteer computing projects. In most instances, the computing devices involved are not located in a controlled environment, like a datacenter, but rather in the private homes or workplaces of enthusiasts. It's inevitable that some of these devices may be faulty, prone to overheating, or inadequately protected from environmental factors such as lightning or moisture.

The conventional approach to identifying such faulty devices involves implementing a ‘double-check’ procedure, where the same primality test is executed by two unrelated computing devices, and their results are subsequently compared. However, this strategy requires twice the amount of work than is strictly necessary. This has led to a persistent demand for more efficient approaches. One alternative involves assessing the reliability of new computing devices, where the double-check is only applied to the initial few tests, after which the device is deemed trustworthy. But this approach cannot guard against intermittent faults or chronic failures, such as those caused by overheating during hot weather or accidental air conditioning shutdowns. Such an incident occurred in 2014 during the primality test of the big number $281 \cdot 2^{2051865} + 1$. Despite being considered reliable, the computing device conducting the test incorrectly identified the number as composite. It wasn't until 2022, during a recheck of suspected data, that the number was found to be prime.

To address this issue, Gerbicz proposes a method for a mathematically rigorous verification of exponentiation operations with exponents in the form of 2^n [7]. This approach can be employed in the Fermat primality test of Mersenne numbers, Proth numbers and “Riesel” numbers. This method not only helps detect hardware failures but also identifies software errors in the implementation of operations involving big numbers. The additional calculations introduced by this method generally result in an overhead of no more than 1%, which is considerably more efficient than a complete double-check.

The objective of the Gerbicz check is to establish the accuracy of each sequence of L squarings. Let's assume the following values have been computed:

$$\begin{aligned} a^{2^L} &= (a)^{2^L} \\ a^{2^{2L}} &= (a^{2^L})^{2^L} \end{aligned} \tag{2.2.1}$$

To verify the correctness of these computations, one could redo them, involving a total of $2L$ squarings. However, by multiplying the two equations, we arrive at:

$$a^{2^L} \cdot a^{2^{2L}} \stackrel{?}{=} (a \cdot a^{2^L})^{2^L} \tag{2.2.2}$$

By performing L squarings, both values can be verified simultaneously, resulting in a 50% reduction in effort. Notably, even if an error occurs during the calculation of a^{2^L} , it can still be used on the right-hand side of (2.2.2). Although both sides of the equation consist of products, each term independently verifies its respective calculation component, irrespective of the other. If the right-hand side of (2.2.2) equals the left-hand side, it strongly indicates that all calculations were executed without errors. The probability of two computation errors offsetting each other is exceedingly low, except in degenerate cases where the common term is equal to 0.

We can check an arbitrary amount of – say, k – exponentiation steps at the same time. In this case the first $k - 1$ terms on the left-hand side are identical to the last $k - 1$ terms on the right-hand side (fig. 2.1). If we designate those terms as

$$d = \prod_{i=1}^{k-1} a^{2^{iL}} \bmod m \tag{2.2.3}$$

Then Gerbicz check can be formulated as

$$d \cdot a^{2^{kL}} \bmod m \stackrel{?}{=} (a \cdot d)^{2^L} \bmod m \tag{2.2.4}$$

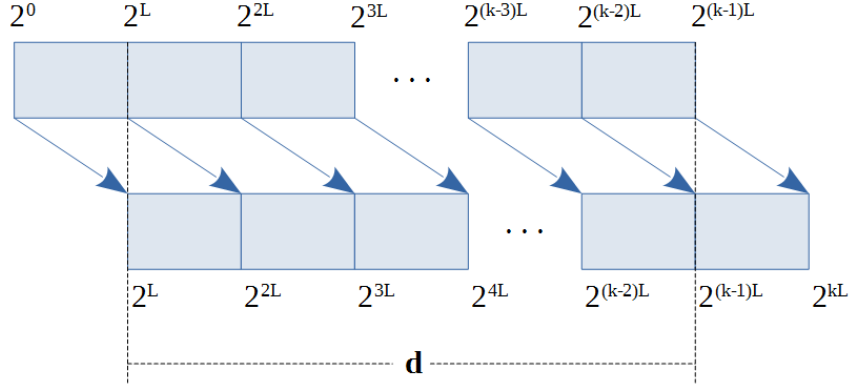


Figure 2.1: Gerbicz check

This approach allows us to avoid storing all intermediate values $a^{2^{iL}}$ in memory and instead accumulate them in $\mathbf{d}$ as the exponentiation process unfolds. From a performance perspective, the check introduces an overhead of k additional multiplications and L additional squarings. Assuming the cost of these operations is similar in this context, the values of k and L should be chosen to be close when configuring the check's parameters. Gerbicz suggests setting k equal to L , but in practical scenarios, their product often needs to be adjusted to a predetermined value, which can be achieved more accurately if they are distinct.

In the event that the Gerbicz check fails, all calculations must be recomputed because pinpointing the exact location of the error is not possible. This is rather inconvenient, as no one wants to lose the entirety of their progress. To address this, some intermediate values are checkpointed to disk. The Gerbicz check is carried out before each checkpoint, and if it fails, the computation is reverted to the previous checkpoint, which is known to be correct. This slightly increases the overhead but significantly enhances convenience.

For instance, consider the previously mentioned number $281 \cdot 2^{2051865} + 1$. The Proth primality test necessitates the evaluation of the following condition:

$$3^{281 \cdot 2^{2051864}} \equiv -1 \pmod{281 \cdot 2^{2051865} + 1}$$

In this case, $a = 3^{281}$ is our base for the exponentiation. This value is not considered a big number by our standards; it can be computed using various multiplication methods to ensure its accuracy. The primary objective is to perform squaring of this value 2051864 times. For the sake of convenience, we establish 10 checkpoints, implying that a Gerbicz check should be conducted after every 205186 squarings. If we opt for equal values of k and L , the closest value would be $L^2 = 452^2 = 204304$. Following 10 checkpoints, we would accumulate 2043040 squarings, leaving a “tail” of 8824 operations. Another Gerbicz check with reduced parameters (and higher overhead) can be employed for this “tail.” However, if we choose $k = 448$ and $L = 458$, their product $L_2 = 448 \cdot 458 = 205184$ results in a “tail” of merely 24 operations. The overhead (excluding the “tail”) amounts to $(448 + 458) \cdot 10 = 9060$ operations, or 0.44%.

2.3 Atnashev certificate

Unfortunately, the Gerbicz check alone is not enough to entirely replace the double check procedure in distributed computing. For one, the determination of whether the exponentiation has successfully passed the check or not is executed by the same computer that performs the calculation. In the event of a severe malfunction, there exists the potential for a false positive outcome. Furthermore, even if an error is correctly detected, reverting to a stored checkpoint has the potential to compromise data integrity, e.g. in the process of copying from an overheating memory module.

For another, unethical participants still pose a risk. The motivations for sabotaging a distributed project can vary, ranging from inflating ratings or deceiving reputation systems to undermining the credibility of a

project to unprovoked digital mischief. In either case, replacing a full double check would only be conceivable if a method were in place to rigorously substantiate the accuracy of claimed computational results.

A slightly modified version of the Gerbicz check can serve as such a method. The key modification revolves around not accumulating the intermediate values within $\mathbf{d}$, but instead storing them as checkpoints $\mathbf{r}_i$. These checkpoints are then uploaded to a trusted server for verification. While computing the checkpoints, participants can apply their own local Gerbicz check to validate the accuracy of each individual checkpoint (similar to the example in the preceding section).

To prevent any confusion between the parameters of distinct checks, we denote the total number of checkpoints as ℓ , and the number of squarings within a single checkpoint as M , chosen in such a way that $\ell M \approx n$. Thus, we have:

$$\mathbf{r}_i = \mathbf{a}^{2^{iM}} \bmod \mathbf{m} \quad (2.3.1)$$

The trusted server can conduct the Gerbicz check by substituting these checkpoints within expressions (2.2.3) and (2.2.4), thereby affirming the correctness of the final checkpoint $\mathbf{r}_\ell$. Assuming the parameters are aptly chosen, the server can utilize $\mathbf{r}_\ell$ to rapidly derive the outcome of the primality test and compare it against the participant's reported result, effectively executing a double check of the "tail."

Unfortunately, this approach has significant drawbacks. Most importantly, it demands great computing capacity of the trusted server. A distributed computing project could potentially involve thousands of participants and tens of thousands of computing devices. Consequently, the trusted server might have to validate hundreds of thousands of primality tests on a daily basis. In order to handle such a substantial workload, the project is compelled to select an exceedingly small value for M . However, as M decreases, the value of ℓ increases. Each checkpoint is a residue modulo the big number, sometimes reaching up to tens of megabytes, so ℓ , the number of checkpoints that participants in a public distributed project can upload to the server, is often capped at a few dozen. This in turn imposes a lower limit on M . Given that each verification task necessitates executing M squarings, the administration of a distributed computing project must possess considerable computational power themselves: $1/\ell$ of the collective computing power of all participants.

To relieve the load on the trusted server, Atnashev introduces the concept of a 'certificate' [8]. An Atnashev certificate comprises a pair of big numbers, denoted as $(\mathbf{x}, \mathbf{y})$, derived from the checkpoints by the trusted server. Here, $\mathbf{x}$ constitutes the public component, while $\mathbf{y}$ serves as the confidential counterpart. Any entity – whether it be the administration, a participant, or a third party – can conduct M squarings on $\mathbf{x}$ and transmit the outcome to the trusted server. If the outcome corresponds to $\mathbf{y}$, the certificate is deemed validated, and the associated primality test is considered verified. Through the implementation of certificates, the trusted server can delegate a substantial portion of the verification process to the participants themselves. The supplementary effort required from participants is merely $1/\ell$ of the principal workload.

Furthermore, the central server does not need to store $\mathbf{y}$ in its entirety, but rather only a hash, effectively only needing to store $\mathbf{x}$. Consequently, the process of certificate validation on the server becomes a straightforward comparison of two hashes.

The certificate is computed by the trusted server from the uploaded checkpoints and a set of random "security powers" e_i :

$$\mathbf{r}_0 = \mathbf{a}, \quad e_i = \text{random}(), \quad \mathbf{x} = \prod_{i=0}^{\ell-1} \mathbf{r}_i^{e_i} \bmod \mathbf{m}, \quad \mathbf{y} = \prod_{i=1}^{\ell} \mathbf{r}_i^{e_{i-1}} \bmod \mathbf{m} \quad (2.3.2)$$

The starting value $\mathbf{r}_0$ is the cornerstone of the proof, and therefore should be computed by the trusted server on its own. The security powers e_i serve several functions. One of them is making checkpoints independent from each other, preventing attacks like

$$\prod_{i=1}^{\ell-1} \mathbf{r}_i \equiv 1 \pmod{\mathbf{m}}, \quad \mathbf{r}_\ell = \mathbf{a}^{2^{\ell M}} \bmod \mathbf{m} \quad (2.3.3)$$

If e_i are predictable, one malicious participant could construct such r_i with a relative ease from random values and the inverse of their product, but securely generated e_i subverts this attack, effectively partitioning the effect of all checkpoints.

$$r_{i+1}^{e_i} \stackrel{?}{\equiv} (r_i^{e_i})^{2^M} \pmod{m} \quad (2.3.4)$$

As a whole, certificate validation works very much like Gerbicz check (fig. 2.2).

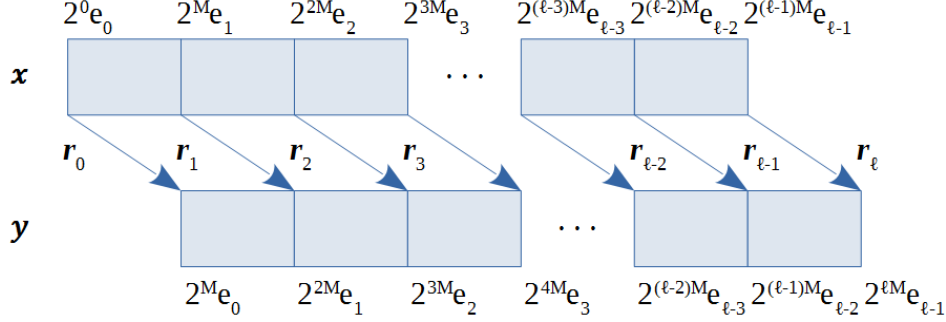


Figure 2.2: Certificate validation

Another function of e_i is the “encryption” of the certificate. This becomes necessary when the validation task is assigned to the same participant who executed the primality test. In such cases, if the participant has malicious intentions or simply wants to save time, they could potentially replay all the operations performed by the server and obtain y without having to do M squarings if e_i are predictable. Therefore, the generation of e_i should be treated as the generation of a secret key.

The described protocol allows us to verify the correctness of the performed calculations. However, the primary objective of any computation is to produce a result. In the case of the Fermat primality test, there are two possible results: equal to 1 (indicating the number is prime) and not equal to 1 (indicating the number is composite). If the result is not equal to 1, the exact value is of no concern, so any attack that alters the value is inconsequential. On the other hand, if the number is prime, it becomes important to investigate the possibility of an attack that replaces 1 with some other value.

Unfortunately, such attack exists. Malicious participant can multiply r_ℓ by -1 (if $\ell M = n$) or by k -th primitive root of unity. If $e_{\ell-1}$ is divisible by k , then during construction of the certificate the root of unity turns into 1, and the certificate will be successfully validated. But the corrupted r_ℓ may produce the result of the test not equal to 1 (for example, -1 in the simplest case). The big number may be erroneously declared composite. The motivation of the participant can be a simple hooliganism, but also a desire to be the only individual in the world owning the prime number. Such an attack is possible because finding primitive roots of unity modulo prime number is pretty easy, especially for numbers of $k \cdot 2^n + 1$ form ($\varphi(m) = k \cdot 2^n$). If the prime number has some other form, one can try finding small divisors of $\varphi(m)$, especially since the probability of success is higher for low power roots of unity.

The same approach can be used to defend against this attack. While checking the result of the test, the trusted server may raise it to powers of small divisors of $\varphi(m)$. In case any of the values happens to be 1, it should be treated as a cause for concern at least.

2.4 Pietrzak function

To fully leverage the optimization that the Gerbicz check under the Atnashev certificate provides would demand ridiculous bandwidth of the trusted server. An opportunity to reduce this data burden was unexpectedly discovered in a different context. Pietrzak introduced a verifiable delay function (VDF) [9] as an

alternative to cryptocurrency mining. The Pietrzak function involves taking a number to the power of 2^n and offers proof of the computation performed. This proof is established using a method closely resembling the one explained earlier, albeit with some notable distinctions.

The Pietrzak function necessitates that ℓ is a power of 2, and the values of e_i are determined by a deterministic function that depends, among other factors, on a hash of the last checkpoint $\mathbf{r}_\ell$. Finally, the deterministic nature of e_i enables the calculation of partial products of the checkpoints on the participant's side. This, in turn, reduces the volume of data that needs to be uploaded to the trusted server from ℓ to $\log_2 \ell$ big numbers (residues).

The Pietrzak function is a recursive algorithm in the form of an exchange between two parties, manipulating a pair of big numbers $(\mathbf{x}_i, \mathbf{y}_i)$ on each iteration. The goal is to determine whether the following expression holds true:

$$\mathbf{y}_i \stackrel{?}{=} \mathbf{x}_i^{2^T} \bmod \mathbf{m} \quad (2.4.1)$$

The verifier can verify the expression explicitly, by performing T squarings. But note that if we multiply both sides with a “middle” value defined as

$$\mathbf{p}_i = \mathbf{x}_i^{2^{T/2}} \bmod \mathbf{m} \quad (2.4.2)$$

then the resulting pair of numbers $(\mathbf{x}_i \cdot \mathbf{p}_i, \mathbf{p}_i \cdot \mathbf{y}_i)$ satisfies (2.4.1) with half the exponent only if the original pair of numbers satisfies (2.4.1), and thus the verification can be done in half the squarings and half the time. Similar to the security powers in the Atnashev certificate, to isolate terms from each other, one of them is raised to a power calculated by hashing current values of all variables. We get the following algorithm:

Algorithm 2.3: Pietrzak function.

```

1   $\mathbf{x}_0 \leftarrow \mathbf{a}, \mathbf{y}_0 \leftarrow \mathbf{r}_\ell, T \leftarrow \ell M$ 
2  for  $i \leftarrow 0$  to  $\log_2 \ell - 1$  do
3       $T \leftarrow T/2$ 
4       $\mathbf{p}_i \leftarrow \mathbf{x}_i^{2^T} \bmod \mathbf{m}$ 
5       $h_i \leftarrow \text{hash}(\mathbf{x}_i, \mathbf{p}_i, \mathbf{y}_i)$ 
6       $\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i^{h_i} \cdot \mathbf{p}_i \bmod \mathbf{m}$ 
7       $\mathbf{y}_{i+1} \leftarrow \mathbf{p}_i^{h_i} \cdot \mathbf{y}_i \bmod \mathbf{m}$ 
8  end
```

// $\mathbf{p}_i = \mathbf{x}_i^{2^T}, \mathbf{y}_i = \mathbf{p}_i^{2^T}$

Note that one does not need to actually perform exponentiation on $\mathbf{x}_i$ to get $\mathbf{p}_i$, because $\mathbf{p}_i$ is a product of 2^i checkpoints:

$$\mathbf{p}_i = \prod_{\substack{j=1 \\ k=j\ell/2^i - \ell/2^{i+1}}}^{2^i} \mathbf{r}_k^{e_k} \quad e_k = \prod_{t=0}^{i-1} \text{bit}(2^i - j, i - t - 1) h_t \quad (2.4.3)$$

For example, if we take $\ell = 8$, the following table shows values obtained at each iteration of the Pietrzak function:

i	T	$\mathbf{p}_i$	$\mathbf{x}_{i+1}$	$\mathbf{y}_{i+1}$
0	$4M$	$\mathbf{r}_4$	$\mathbf{r}_0^{h_0} \mathbf{r}_4$	$\mathbf{r}_4^{h_0} \mathbf{r}_8$
1	$2M$	$\mathbf{r}_2^{h_0} \mathbf{r}_6$	$\mathbf{r}_0^{h_0 h_1} \mathbf{r}_2^{h_0} \mathbf{r}_4^{h_1} \mathbf{r}_6$	$\mathbf{r}_2^{h_0 h_1} \mathbf{r}_4^{h_0} \mathbf{r}_6^{h_1} \mathbf{r}_8$
2	M	$\mathbf{r}_1^{h_0 h_1} \mathbf{r}_3^{h_0} \mathbf{r}_5^{h_1} \mathbf{r}_7$	$\mathbf{r}_0^{h_0 h_1 h_2} \mathbf{r}_1^{h_0 h_1} \mathbf{r}_2^{h_0 h_2} \mathbf{r}_3^{h_0} \mathbf{r}_4^{h_1 h_2} \mathbf{r}_5^{h_1} \mathbf{r}_6^{h_2} \mathbf{r}_7$	$\mathbf{r}_1^{h_0 h_1 h_2} \mathbf{r}_2^{h_0 h_1} \mathbf{r}_3^{h_0 h_2} \mathbf{r}_4^{h_0} \mathbf{r}_5^{h_1 h_2} \mathbf{r}_6^{h_1} \mathbf{r}_7^{h_2} \mathbf{r}_8$

Note that each $\mathbf{r}_j$ in $\mathbf{x}_{i+1}$ has a unique e_j ; verification of $(\mathbf{x}_3, \mathbf{y}_3)$ is identical to the operation shown on fig. 2.2.

Employing recursion opens up some additional useful features. For example, one can choose n as the starting value of T even if it is not divisible by M . It means that at some step of the recursion T becomes odd. To handle that situation correctly, the following line should be added to the beginning of each step:

if $odd(T)$ **then** $x_i \leftarrow x_i^2 \bmod m, T \leftarrow T - 1$

This operation “moves” x_i by one position, so that T becomes even. As a result, the checkpoints necessary to calculate p_i become positioned at irregular intervals. It introduces some complexity to their calculation, but eliminates the “tail” completely, r_ℓ becomes the result of the primality test for all n .

The trusted server can replay the recursion having only the last checkpoint r_ℓ and all partial products p_i . If any of p_i is incorrect, meaning that it’s not a “midpoint” between x_i and y_i , then the produced pair will not satisfy (2.4.1) and the verification operation will fail. At the same time, if r_ℓ is not the result of exponentiation, the verification operation will fail too. Unfortunately, both statements are not entirely correct because the roots of unity attack can be performed even easier since e_j are deterministic. Malicious participant can spend time multiplying checkpoints by a root of unity until the verification operation has equal powers of the root at both sides of the check. Only then the corrupted r_ℓ and corrupted p_i get uploaded. This makes checking for the corruption mandatory on the trusted server.

The pair of big numbers $(x_{\log_2 \ell}, y_{\log_2 \ell})$ produced at the end of the algorithm can act as Atnashev certificate for public verification. But since Pietrzak function is deterministic, the certificate is deterministic too. The participant, who performed the exponentiation and calculated partial products using the function, knows the secret part of the certificate, which undermines the whole idea. As a solution, Atnashev suggests to perform explicit “encryption” of the certificate:

$$e = \text{random}(), \quad x' = x^e \bmod m, \quad y' = y^e \bmod m \quad (2.4.4)$$

The secret key e should be long enough to prevent calculation of the discrete logarithm in a reasonable time, even if $\varphi(m)$ is composed of small divisors.

At the same time, Pietrzak function can be used alone, without any certificate. The algorithm allows to decrease T until it becomes so small that the trusted server can perform the verification operation on its own, without the need to offload the task via certificate. But the peculiarities of big numbers show up again. Although participants need to upload only $\log_2 \ell$ big numbers per test, they need to store all ℓ checkpoints locally, because the partial products can start to be calculated only after the last checkpoint is obtained. And while storing of thousands of checkpoints several megabytes each is not a problem in the modern world, storing of millions of such checkpoints is troublesome. This limits the depth of the recursion and makes standalone usage of Pietrzak function impractical. A combination of Pietrzak function with Atnashev certificate (and Gerbicz check of each checkpoint) is the most versatile and reliable tool in practice.

2.5 Non-binary exponent

The attentive reader may have noticed that all expressions given in previous sections stay correct even if base 2 in exponents is replaced with arbitrary base b . Therefore, all algorithms and verification methods can be used for exponents of b^n form. Unfortunately, practical implementation of such replacement meets with some difficulties.

The main problem is that it is necessary to perform big number multiplication operations in addition to big number squarings to support verification of exponentiation with b^n exponents, even if the base of exponentiation is a small constant $a = 3$. This may look strange at first, because as we stated above, left-to-right algorithm allows to replace big number multiplication with small constant multiplication. However, all verification methods are based on making use of intermediary values, obtained during continuous exponentiation, as in (2.2.1). For example, to perform verification of raising to $5^4 = 625$ power, one needs to keep

the intermediary value with $5^2 = 25$ exponent. By using left-to-right algorithm, raising to $5^4 = 625$ power is performed by 10 iterations:

j	9	8	7	6	5	4	3	2	1	0
e_j	1	0	0	1	1	1	0	0	0	1
$\mathbf{r}$	$\mathbf{a}$	$\mathbf{a}^2$	$\mathbf{a}^4$	$\mathbf{a}^9$	$\mathbf{a}^{19}$	$\mathbf{a}^{39}$	$\mathbf{a}^{78}$	$\mathbf{a}^{156}$	$\mathbf{a}^{312}$	$\mathbf{a}^{625}$

No intermediary value has exponent 25. Furthermore, raising to 25-th power gives a completely different set of intermediary values:

j	4	3	2	1	0
e_j	1	1	0	0	1
r	a	a^3	a^6	a^{12}	a^{25}

Therefore, both of those operations can't be performed simultaneously. To compute $\mathbf{a}^{5^4}$ out of $\mathbf{a}^{5^2}$, one needs to raise base $\mathbf{a}' = \mathbf{a}^{5^2}$ to 5^2 power. And base $\mathbf{a}'$ is not a small constant anymore.

As a result, Fermat primality test of a number of $b^n + 1$ form performed with Gerbicz check turns out to be 50% slower than without the check. Fortunately, the situation can be significantly improved by implementing one of the exponentiation algorithms which minimize the number of multiplications [6]. The most suitable one in our case is the sliding window method.

The main idea of the sliding window method of length W is to split exponent e in two parts. One part is simply a power of two, the other part contains bit substrings of no more than W bits, with the least significant bit of each substring equal to 1. For example, $e = 11957708941720303968251$ is split into the following parts:

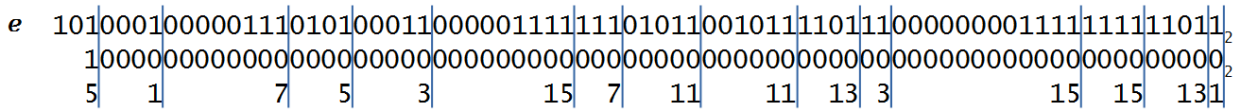


Figure 2.3: Sliding window $W = 4$.

The raising to a power of two is performed from left to right, while multiplication by powers of non-zero substrings is performed by employing a dictionary. The precomputed dictionary should contain 2^{W-1} big numbers, because only odd powers need to be stored there. The choice of W depends on the amount of memory available for the dictionary, but also on the amount of operations necessary to populate the dictionary. There's no point wasting time computing a big dictionary for a relatively small e , even if there's enough memory to store it.

The method turns out to be faster than a simple left-to-right exponentiation because the multiplication operation is performed at most once per W squarings, and for arbitrary exponents even once per $W + 1$ squarings on average. In practical applications, considering the size of big numbers and the available memory, W typically belongs to 5—10 range. This means that for arbitrary exponents the number of multiplication operations is 10-20% of the number of squarings. Therefore, by employing the sliding window method Fermat primality test with Gerbicz check becomes just 10-20% slower than without the check.

Algorithm 2.4: Exponentiation with sliding window of length W .

Output: $r = a^e \bmod m$

```

1   $d_0 \leftarrow a, \quad s \leftarrow a^2 \bmod m$ 
2  for  $i \leftarrow 1$  to  $2^{W-1} - 1$  do
3       $d_i \leftarrow s \cdot d_{i-1} \bmod m$ 
4  end
5   $r \leftarrow 1$ 
6  for  $j \leftarrow \lfloor \log_2 e \rfloor$  to 0 do
7      if  $e_j = 0$  then
8           $r \leftarrow r^2 \bmod m$ 
9      else
10          $i \leftarrow \max(j - W + 1, 0)$ 
11         while  $e_i = 0$  do  $i \leftarrow i + 1$ 
12          $u \leftarrow 0$ 
13         while  $j > i$  do
14             if  $r \neq 1$  then  $r \leftarrow r^2 \bmod m$ 
15              $u \leftarrow 2u + e_j$ 
16              $j \leftarrow j - 1$ 
17         end
18         if  $r = 1$  then  $r \leftarrow d_u$ 
19         else  $r \leftarrow r \cdot d_u \bmod m$ 
20     end
21 end

```

2.6 Li generalization

By employing the sliding window method one can use strong checks and certificates in Fermat primality tests of numbers of $k \cdot b^n + c$ form. But this approach is not perfect, because it leads to both increase in the number of operations and increase in data transfers to and from slow memory because of the need to access the dictionary. Besides, no algorithm accelerating exponentiation can help to apply strong checks to primality tests of numbers, which can't have the exponent of the Fermat test to be represented as b^n with large enough n . Factorial primes $n! \pm 1$ are an example of such numbers. There's an interest in factorial primes because there are deterministic primality tests for them. But the search for primes of all such forms is hindered by the necessity to perform double check.

As a solution to all these problems Li suggests a method which generalizes for arbitrary exponents all checks and certificates described in this chapter [10]. The main idea of the method is to perform independent checks of squaring operations and multiplication operations in the left-to-right exponentiation. Suppose we are at j -th iteration of algorithm 2.2, we have the following intermediary value:

$$r_j = a^{\lfloor e/2^j \rfloor} \bmod m \quad (2.6.1)$$

Then at $j-L$ iteration (j decreases as the algorithm progresses) the value is squared L times and is multiplied by the base raised to a power which is a combination of all L bits e_j encountered between the iterations, in other words a bit substring of e of length L , which we designate as $e_{[\text{start}, \text{start}+L)}$. The intermediary value at iteration $j-L$ can be written in the following way:

$$r_{j-L} = r_j^{2^L} \cdot a^{e_{[j-L, j)}} \bmod m \quad (2.6.2)$$

The intermediary value at iteration $j - 2L$ can be expressed via the value at iteration $j - L$ in the same way:

$$\mathbf{r}_{j-2L} = \mathbf{r}_{j-L}^{2^L} \cdot \mathbf{a}^{e_{[j-2L, j-L]}} \bmod \mathbf{m} \quad (2.6.3)$$

These two expressions are analogous to expressions (2.2.1). So we can construct an expression analogous to (2.2.2) by multiplying them:

$$\mathbf{r}_{j-L} \cdot \mathbf{r}_{j-2L} \bmod \mathbf{m} \stackrel{?}{=} (\mathbf{r}_j \cdot \mathbf{r}_{j-L})^{2^L} \cdot \mathbf{a}^{e_{[j-L, j]} + e_{[j-2L, j-L]}} \bmod \mathbf{m} \quad (2.6.4)$$

To verify the expression one needs to perform L squarings and also raise the base $\mathbf{a}$ to a power equal to the sum of two bit substrings of length L , that is to a power only slightly larger than 2^L . If the base $\mathbf{a}$ is a small constant, then raising it to such power has a comparable performance as L squarings. If it's not a small constant, then the sliding window method can be used, minimizing time penalty.

The generalization (2.6.4) can be applied directly to Gerbicz check (2.2.4). Accumulation of common terms is performed totally in the same way:

$$\mathbf{d} = \prod_{i=1}^{k-1} \mathbf{r}_{j-iL} \bmod \mathbf{m} \quad (2.6.5)$$

But besides it, the sum $\mathbf{s}$ of bit substrings is accumulated too:

$$\mathbf{s} = \sum_{i=1}^k e_{[j-iL, j-iL+L]}, \quad \mathbf{s} < k2^L \quad (2.6.6)$$

Finally, we can formulate Gerbicz-Li check in the following way:

$$\mathbf{d} \cdot \mathbf{r}_{j-kL} \bmod \mathbf{m} \stackrel{?}{=} (\mathbf{r}_j \cdot \mathbf{d})^{2^L} \cdot \mathbf{a}^{\mathbf{s}} \bmod \mathbf{m} \quad (2.6.7)$$

In practical implementation of the check it is possible to perform both exponentiations at the same time. In order to do so, one just needs to set the following initial values in algorithm 2.2:

$$\mathbf{r} = \mathbf{r}_j \cdot \mathbf{d} \cdot \mathbf{a}^{\lfloor \mathbf{s}/2^L \rfloor}, \quad \mathbf{e} = \mathbf{s} \bmod 2^L, \quad j = L - 1 \quad (2.6.8)$$

When the algorithm proceeds, it performs L squarings and multiplications by the base, producing the result equal to the right side of (2.6.7). This means the overhead of Gerbicz-Li check is comparable to the overhead of Gerbicz check.

The expression (2.6.7) also demonstrates the distinctive feature of the method: Gerbicz-Li check verifies correctness of calculations between checkpoint $\mathbf{r}_j$ and checkpoint $\mathbf{r}_{j-kL}$. It means that the check itself does not force changing the base $\mathbf{a}$. If it is a small constant, then it stays the small constant after the check. Fermat primality test of numbers of arbitrary form can be performed with Gerbicz-Li check almost as fast as without the check.

As an example, let's take raising to the power $\mathbf{e} = 11957708941720303968251$ with $L = 15$, $k = 5$. The exponent can be represented as $\mathbf{e} = 10371 \cdot 2^{60} + 21600 \cdot 2^{45} + 32601 \cdot 2^{30} + 15808 \cdot 2^{15} + 8187$. Take $j = 75$, $\mathbf{r}_{75} = 1$ as the starting values. Then we get the following intermediary values:

$$\begin{aligned} \mathbf{r}_{60} &= \mathbf{r}_{75}^{2^{15}} \cdot \mathbf{a}^{10371} \bmod \mathbf{m} \\ \mathbf{r}_{45} &= \mathbf{r}_{60}^{2^{15}} \cdot \mathbf{a}^{21600} \bmod \mathbf{m} \\ \mathbf{r}_{30} &= \mathbf{r}_{45}^{2^{15}} \cdot \mathbf{a}^{32601} \bmod \mathbf{m} \\ \mathbf{r}_{15} &= \mathbf{r}_{30}^{2^{15}} \cdot \mathbf{a}^{15808} \bmod \mathbf{m} \\ \mathbf{r}_0 &= \mathbf{r}_{15}^{2^{15}} \cdot \mathbf{a}^{8187} \bmod \mathbf{m} \end{aligned}$$

Gerbicz-Li check is verifying the following expression:

$$\mathbf{r}_{60} \cdot \mathbf{r}_{45} \cdot \mathbf{r}_{30} \cdot \mathbf{r}_{15} \cdot \mathbf{r}_0 \bmod \mathbf{m} \stackrel{?}{=} (\mathbf{r}_{75} \cdot \mathbf{r}_{60} \cdot \mathbf{r}_{45} \cdot \mathbf{r}_{30} \cdot \mathbf{r}_{15})^{2^{15}} \cdot \mathbf{a}^{88567} \bmod \mathbf{m}$$

Li generalization can be applied in a similar way to Pietrzak function. On each step of the recursion the binary representation of the exponent is split in two halves of equal length, one of them is multiplied by the current hash value and is added to the other. In the end, $\mathbf{s}$ becomes a sum of all bit substrings of length M positioned at the same places as checkpoints $\mathbf{r}_k$, multiplied by the corresponding security powers e_k .

In case T is odd on some step of the recursion, the exponent is split in two halves with lengths differing by 1 bit. But Pietrzak-Li function allows to completely avoid such a situation. Note that in the example above the length of the exponent is 74 bits, but we added the leading 0 to get 75 bits. The starting value in this case is 1. Pietrzak function fails if $\mathbf{x}_0 = 1$, but for Pietrzak-Li function it's a completely normal situation. Let's apply the function to the example above, take $\ell = 4$. For additional reliability, we use Gerbicz-Li check $L = 5$, $k = 4$ while calculating each checkpoint. Thus, checkpoints are saved after every 20 bits, making the exponent $\ell k L = 80$ bits long, out of which 6 leading bits are 0. This makes the first two values of the first Gerbicz-Li check equal to 1.

Atnashev-Li certificate should include $\mathbf{s}$ in the public part, because now the validation operation is

$$\mathbf{y} \stackrel{?}{=} \mathbf{x}^{2^M} \cdot \mathbf{a}^{\mathbf{s}} \bmod \mathbf{m} \quad (2.6.9)$$

But if encryption is used (both sides of the check are raised to a secret power e), then the certificate should include $e\mathbf{s}$, which allows the malicious participant to trivially recover e . The solution is to also include the encrypted base in the public part:

$$\mathbf{a}' = \mathbf{a}^e \bmod \mathbf{m} \quad (2.6.10)$$

The validation of Atnashev-Li certificate can be optimized in the same way as described above, by performing both exponentiation operations at the same time. Because of security concerns $\mathbf{a}'$ should be a big number, therefore it is recommended to use the sliding window method, with the initial values computed similar to (2.6.8).

2.7 Lucas sequences

Exponentiation is the most popular, but not the only operation used in primality tests. Another popular and in some sense similar operation is evaluation of Lucas sequences. Deterministic primality tests of Mersenne numbers and numbers of $k \cdot b^n - 1$ form use Lucas sequences, and in some cases their performance is very close to performance of probabilistic Fermat test which uses exponentiation.

Lucas sequences are a pair of recursive integer sequences V_n and U_n with parameters P and Q , with discriminant $D = P^2 - 4Q \neq 0$, satisfying the recurrence relations:

$$V_0 = 2, \quad V_1 = P, \quad V_n = PV_{n-1} - QV_{n-2} \quad (2.7.1)$$

$$U_0 = 0, \quad U_1 = 1, \quad U_n = PU_{n-1} - QU_{n-2} \quad (2.7.2)$$

By choosing specific P and Q , one can get many famous sequences like Fibonacci numbers, Lucas numbers, Mersenne numbers and even Chebyshev polynomials of first and second kinds. But in the context of big numbers, we're mostly interested in specific cases which allow to compute terms of the sequences using as few multiplication operations on big numbers as possible. This is aided by many relations involving terms of Lucas sequences. For example, the sequence V_n satisfies the following relations:

$$V_{2n} = V_n^2 - 2Q^n \quad (2.7.3)$$

$$V_{m+n} = V_m V_n - Q^n V_{m-n} \quad (2.7.4)$$

$$V_{mn} = V_m(P = V_n, Q = Q^n) \quad (2.7.5)$$

This sequence is used in Lucas–Lehmer primality test of Mersenne numbers $\mathbf{m} = 2^p - 1$. The test requires calculation of $V_{2^{p-2}} \pmod{\mathbf{m}}$ with $P = 4$ and $Q = 1$. It can be done by applying (2.7.3) $p - 2$ times. When $Q = 1$ that operation is reduced to a simple squaring operation and subtraction of 2, which can be done in the normalization stage for free. Therefore, if we only consider operations on big numbers, the test is practically indistinguishable from Fermat test, where one needs to perform $p - 1$ squarings. It's exactly thanks to this similarity we examine Lucas sequences in this chapter.

In the same way exponentiation becomes complicated when the exponent is not a power of two, calculation of $V_e \pmod{\mathbf{m}}$ poses a challenge for arbitrary e . It's not possible to use an algorithm analogous to left-to-right exponentiation because that algorithm would require an operation which increments the index of a term by 1 (an analogue of multiplication by $\mathbf{a}$). But as follows from (2.7.1), to compute the next term one needs to know the previous term, which is not possible in general case. One of the simplest solutions to this problem is application of the method called ‘Montgomery ladder’ [11]. The algorithm is somewhat similar to left-to-right exponentiation, but in each iteration operations are performed on a pair of terms (V_n, V_{n+1}) . Depending on the value of bit e_j , either (V_{2n}, V_{2n+1}) or (V_{2n+1}, V_{2n+2}) are calculated using (2.7.3) and (2.7.4). Note that the condition that the second index is one higher than the first index holds in both cases. At the end of the algorithm the first term is equal to V_e .

Algorithm 2.5: Montgomery ladder.

Output: $V = V_e \pmod{\mathbf{m}}$

```

1    $V \leftarrow 2$ 
2    $V_{+1} \leftarrow P$ 
3   for  $j \leftarrow \log_2 e$  to 0 do
4       if  $e_j \neq 0$  then
5            $V \leftarrow V_{+1} \cdot V - P \pmod{\mathbf{m}}$                 //  $n' = (n + 1) + n = 2n + 1$ 
6            $V_{+1} \leftarrow V_{+1}^2 - 2 \pmod{\mathbf{m}}$                 //  $n' + 1 = 2(n + 1) = 2n + 2$ 
7       else
8            $V_{+1} \leftarrow V_{+1} \cdot V - P \pmod{\mathbf{m}}$             //  $n' + 1 = (n + 1) + n = 2n + 1$ 
9            $V \leftarrow V^2 - 2 \pmod{\mathbf{m}}$                         //  $n' = 2n$ 
10      end
11  end
```

Each iteration of the algorithm performs the same two operations: multiplication with subtraction and squaring with subtraction of a small constant. Multiplication with subtraction can be implemented as a nested operation (see section 1.10). Thus, the performance of algorithm 2.5 is comparable to the performance of right-to-left exponentiation when all $e_i = 1$. But while the exponentiation algorithms speed up with a low number of non-zero bits in the exponent, the performance of Montgomery ladder is insensitive to the exact values of bits in e . This is the main disadvantage of the method.

Application of methods like the sliding window method is also not possible because to calculate V_{m+n} term using (2.7.4) one needs to know V_{m-n} term. This kind of operations is known as ‘differential addition’. Sequences, in which each next term is obtained via differential addition of previous terms, are called ‘differential addition chains’ (DAC). It's easy to see that Montgomery ladder creates such a chain in the course of its work. For example, for $e = 97$ the resulting chain can be expressed as

$$\text{ladder}(97) = 0, 1, 1, 2, 3, 4, 6, 7, 12, 13, 24, 25, 48, 49, 97, 98.$$

Here and later we list only the indexes of terms comprising the chain.

The problem of finding a differential addition chain of minimal length turns out to be quite challenging. There are many algorithms developed to solve it: Euclid's chain, Bleichenbacher's chain, Tsuruoka's chain, Montgomery PRAC [12, section 4.8]. As an example, let's take a simple but quite useful in practice S-chain.

S-chain is defined recursively as a function of two variables:

$$S(d, e) = \begin{cases} 0, e & d = 0 \\ S(e - d, e) & e/2 < d \\ S(d, e/2), e - d, e & 0 < d < e/4, e \in 2\mathbb{Z} \\ S(d, e - d), e & \end{cases} \quad (2.7.6)$$

The function is called with the second variable set to the target index, the first variable set to some initial difference. For example, for the same $e = 97$ and initial difference $d = 11$ the resulting chain is

$$S(11, 97) = 0, 1, 2, 3, 4, 5, 9, 10, 11, 21, 32, 43, 75, 86, 97.$$

It's easy to see that this chain provides no advantage over the Montgomery ladder. The trick is in the choice of initial difference d . It is recommended to try several values of d near the golden ratio of e to find the chain of optimal length. More complicated algorithms like Montgomery PRAC find shorter chains on average for such d (fig. 2.4). But even for PRAC the probability of optimal chain never reaches 100%. It means that it is possible to find a shorter chain if one has the ability to look through all possible d for the given e and choose the best one. In such a case, a more complicated algorithm may even be worse than a simpler algorithm. For example, for primes $< 10^6$ the optimal S chain is never longer than the optimal PRAC chain, and in a few cases is even shorter. For $e = 97$ the optimal $d = 21$:

$$S(21, 97) = 0, 1, 2, 3, 5, 8, 13, 21, 34, 55, 76, 97.$$

In other words, the choice of differential addition chain algorithm depends among other things on the ability to precompute the optimal d for the range of all possible e . In practical applications, like Morrison primality test or some factorization methods, e is often smooth, that is, a product of many small prime divisors. By applying (2.7.5), one can reduce calculation of $V_e \pmod{m}$ to a successive evaluation of differential addition chains corresponding to the small primes. The list of optimal d for S-chains of primes $< 10^7$ can be found in the source code of PRST utility.

Considering the performance, application of differential addition chains allows to calculate $V_e \pmod{m}$ using $\sim 1.5 \log_2 e$ multiplications of big numbers on average, which is comparable to the performance of right-to-left exponentiation for arbitrary e . Of course the performance can't be improved further with the sliding window method or the optimization of multiplication by a small constant. But in cases when one does not care about performance at all, an interesting outcome can be achieved by calculating both Lucas sequences (U_n, V_n) at once, by using the following relations:

$$U_{2n} = U_n V_n \quad (2.7.7)$$

$$2U_{n+1} = V_n + P U_n \quad (2.7.8)$$

$$2V_{n+1} = P V_n + D U_n \quad (2.7.9)$$

$$2U_{m+n} = U_m V_n + V_m U_n \quad (2.7.10)$$

$$2V_{m+n} = V_m V_n + D U_m U_n \quad (2.7.11)$$

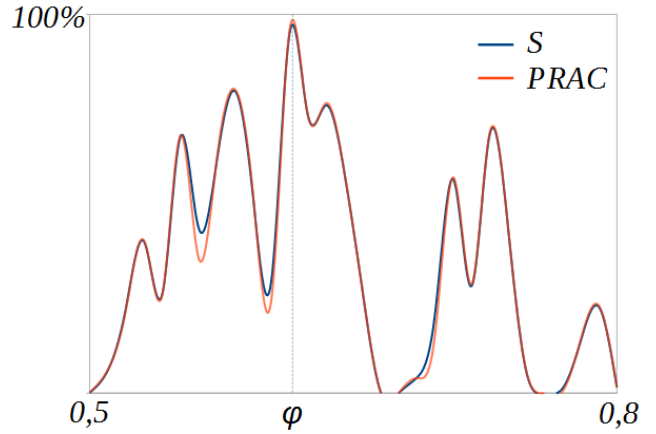


Figure 2.4: The probability of optimal d . PRAC is better near the golden ratio.

These relations provide operations of index doubling, incrementing index by 1, ordinary, not differential, addition of two pairs. And it means that all algorithms described in this chapter become applicable to (U_n, V_n) pair. Finally one can use the sliding window method. And Morrison primality test can make use of Gerbicz-Li check to improve reliability and credibility of its deterministic verdicts.

2.8 Edwards curves

Operations of exponentiation modulo a big number and evaluation of Lucas sequences modulo a big number are interesting to researchers because when repeated many times over, sooner or later the resulting values start to recur (they form a cyclic group). By analyzing the recurrence interval (the order of the group), some statements about the nature of the big number can be made. Many primality tests and factorization methods are based on specific properties of group orders. But exponentiation and Lucas sequences are not the only operations producing cyclic groups. There are many other mathematical objects and operations which can be used in algorithms analyzing group orders. In some cases the same algorithm can be executed several times using different operations and different groups, until a suitable cyclic group is found.

Algebraic groups are an example of such mathematical objects. An algebraic group is composed of sets of numbers satisfying some algebraic equations, like $y = kx + b$. Since it's a group, there is an "addition" operation defined on its elements, in the same sense as the operation of index addition is defined for Lucas sequences. The group can be cyclic if the addition operation has all the necessary properties, like being commutative. The branch of mathematics which studies algebraic groups is called 'algebraic geometry', that's why geometric terminology is often used to describe such groups. Of course, if one performs all operations modulo a big number, then all geometric sense is lost, but the terminology stays the same.

The "line" $y = kx + b \bmod m$ with a simple vector addition modulo m is a cyclic group, but not particularly interesting. Its order is always equal to m . So-called 'elliptic curves' are a much more interesting kind of groups. They are a rather complicated objects used in many branches of mathematics. An elliptic curve can be described with an algebraic equation of degree three, which after a few simple coordinate transforms can be converted into the Weierstrass normal form:

$$y^2 = x^3 + ax + b \quad (2.8.1)$$

The operation of addition of two inequal elements $(x_1, y_1) + (x_2, y_2) = (x_3, y_3)$ is defined as

$$\begin{aligned} \lambda &= \frac{y_2 - y_1}{x_2 - x_1}, \\ x_3 &= \lambda^2 - x_1 - x_2, \\ y_3 &= \lambda(x_1 - x_3) - y_1. \end{aligned} \quad (2.8.2)$$

For doubling of an element $2(x_1, y_1) = (x_3, y_3)$ a separate operation is defined:

$$\begin{aligned} \lambda &= \frac{3x_1^2 + a}{2y_1}, \\ x_3 &= \lambda^2 - 2x_1, \\ y_3 &= \lambda(x_1 - x_3) - y_1. \end{aligned} \quad (2.8.3)$$

Those two operations are enough to define a "multiplication" operation which multiplies a group element by a natural number. It can be implemented by the sliding window method, among others. Unfortunately, the formulas for the λ parameter require a division operation, which can be substituted by multiplication by the modular inverse, computed using the extended Euclidean algorithm. For big numbers such an operation is relatively slow and becomes a bottleneck. One of the ways to speed up the computation is to use "projective"

coordinates (X, Y, Z) defined as

$$\begin{aligned} (X, Y, Z) : Y^2 &\equiv X^3 + aXZ^4 + bZ^6 \pmod{m}, \\ x &= \frac{X}{Z^2} \pmod{m}, \quad y = \frac{Y}{Z^3} \pmod{m}. \end{aligned} \quad (2.8.4)$$

It is not hard to rewrite addition and doubling operations in projective coordinates. In this way the addition operation requires 16 multiplications and no division at all. By using the sliding window method with projective coordinates, one can obtain the resulting group element relatively fast and compute $Z^{-1} \pmod{m}$ only for it.

Still, the multiplication on elliptic curves is extremely slow compared to any exponentiation algorithm. But as we mentioned above, elliptic curves can be described by many different algebraic equations, not only by the Weierstrass normal form. For other forms the group operations have to be defined in a different way, possibly more suitable for fast computation. Moreover, the operations themselves can be optimized in some cases. For example, Montgomery noticed that factorization algorithms he implemented used only the single coordinate x when analyzing group orders. Therefore any form of elliptic curves that allows to compute addition and doubling operations using only one coordinate provides a clear benefit to such algorithms. He suggested the following form [13]:

$$By^2 = x^3 + Ax^2 + x \quad (2.8.5)$$

Projective coordinates (X, Z) are defined as

$$\begin{aligned} ((X, Z), y) : By^2Z^4 &\equiv X^3Z + AX^2Z^2 + XZ^3 \pmod{m}, \\ x &= \frac{X}{Z} \pmod{m}. \end{aligned} \quad (2.8.6)$$

Montgomery also defined the operation of single-coordinate doubling $2(X, Z)$, requiring 5 multiplications, and the operation of single-coordinate addition $(X_1, Z_1) + (X_2, Z_2)$, requiring 6 multiplications and knowledge of the difference $(X_1, Z_1) - (X_2, Z_2)$. In other words, it's a differential addition operation. The multiplication by natural number operation can be implemented by any of the methods described in the previous section, like Montgomery ladder and differential addition chains (Montgomery PRAC). This form of elliptic curves with single-coordinate operations on elements is called 'Montgomery curve'.

For decades Montgomery curves were the fastest and the most efficient, until a new form was discovered. It was suggested by Edwards and improved by Bernstein and Lange [14].

$$ax^2 + y^2 = 1 + dx^2y^2 \quad (2.8.7)$$

When $a = 1$ the form is called 'Edwards curve', when $a \neq 1$ it is called 'twisted Edwards curve'. Each twisted Edwards curve is birationally equivalent to some Montgomery curve and vice versa. It means that there are simple formulas that allow to convert coordinates from one form to the other. Consequently, Montgomery single-coordinate operations can be implemented on Edwards curves too. Besides them, there are very efficient two-coordinate operations. Edwards curves are appealing exactly because they allow both kinds of operations at the same time.

Projective coordinates (X_M, Z_M) on a Montgomery curve can be converted to projective coordinates (Y_{Ed}, Z_{Ed}) on the corresponding twisted Edwards curve the following way:

$$\begin{aligned} Y_{Ed} &= X_M - Z_M, \\ Z_{Ed} &= X_M + Z_M, \\ \frac{d}{a} &= 1 - \frac{1}{(A+2)/4}. \end{aligned} \quad (2.8.8)$$

Single-coordinate differential addition on twisted Edwards curves $(Y_1, Z_1) + (Y_2, Z_2) = (Y_3, Z_3)$ with difference $(Y_1, Z_1) - (Y_2, Z_2) = (Y_0, Z_0)$ is defined as

$$\begin{aligned} X_M &= (Z_0 - Y_0)(Y_1 Z_2 + Z_1 Y_2)^2, \\ Z_M &= (Z_0 + Y_0)(Y_1 Z_2 - Z_1 Y_2)^2, \\ Y_3 &= X_M - Z_M, \\ Z_3 &= X_M + Z_M. \end{aligned} \tag{2.8.9}$$

Computation of Montgomery coordinates is a necessary intermediate step. The same is true for the single-coordinate doubling $2(Y_1, Z_1) = (Y_3, Z_3)$:

$$\begin{aligned} X_M &= Z_1^2(Y_1^2 - \frac{d}{a}Y_1^2), \\ Y_M &= (Z_1^2 - Y_1^2)(Z_1^2 - \frac{d}{a}Y_1^2), \\ Y_3 &= X_M - Z_M, \\ Z_3 &= X_M + Z_M. \end{aligned} \tag{2.8.10}$$

Those operations are identical to the corresponding operations on Montgomery curves and require the same number of multiplications.

It is convenient to use so-called ‘extended’ coordinates for two-coordinate operations on twisted Edwards curves [15]. They extend projective coordinates by introducing an auxiliary value $T = xyZ$. The elliptic curve equation becomes quite simple in extended coordinates:

$$\begin{aligned} (X, Y, Z, T) : aX^2 + Y^2 &\equiv Z^2 + dT^2 \pmod{m}, \\ x = \frac{X}{Z} \pmod{m}, \quad y = \frac{Y}{Z} \pmod{m}, \quad T &= \frac{XY}{Z} \pmod{m}. \end{aligned} \tag{2.8.11}$$

Addition on twisted Edwards curves $(X_1, Y_1, Z_1, T_1) + (X_2, Y_2, Z_2, T_2) = (X_3, Y_3, Z_3, T_3)$ is defined as

$$\begin{aligned} X_3 &= (X_1 Y_2 - Y_1 X_2)(T_1 Z_2 + Z_1 T_2), \\ Y_3 &= (Y_1 Y_2 + a X_1 X_2)(T_1 Z_2 - Z_1 T_2), \\ Z_3 &= (Y_1 Y_2 + a X_1 X_2)(X_1 Y_2 - Y_1 X_2), \\ T_3 &= (T_1 Z_2 + Z_1 T_2)(T_1 Z_2 - Z_1 T_2). \end{aligned} \tag{2.8.12}$$

Note that each formula is a multiple of two values, with only 4 distinct values in all formulas. Some of the values can be computed with nested operations (see section 1.10). Besides, when $a = 1$ all formulas become completely free of the curve parameters, which is very convenient in practice.

Doubling of an element $2(X_1, Y_1, Z_1, T_1) = (X_3, Y_3, Z_3, T_3)$ is defined as

$$\begin{aligned} X_3 &= 2X_1 Y_1 (2Z_1^2 - Y_1^2 - aX_1^2), \\ Y_3 &= (Y_1^2 + aX_1^2)(Y_1^2 - aX_1^2), \\ Z_3 &= (Y_1^2 + aX_1^2)(2Z_1^2 - Y_1^2 - aX_1^2), \\ T_3 &= 2X_1 Y_1 (Y_1^2 - aX_1^2). \end{aligned} \tag{2.8.13}$$

Once again, each formula is a multiple of two values, with only 4 distinct values in all formulas. Computation of the formulas can be done in several different ways, depending on the available free word space [5]. When $a = 1$ the formulas are free of the curve parameters. Besides, T_1 is not used at all, which means the doubling operation can be applied to elements defined by projective coordinates (X_1, Y_1, Z_1) . It is especially useful when performing several consecutive doublings, as in the sliding window method. All doublings but the last

one can be performed without computing T_3 at all. In such a case, the single doubling operation requires only 7 multiplications (multiplication by 2 is performed in the normalization stage).

The advantage of two-coordinate operations is that the addition operation is ordinary (not differential), which allows to implement multiplication of a group element by a natural number using any appropriate method, like the mentioned sliding window method. Furthermore, there is one more useful feature. If we negate X_2 and T_2 in (2.8.12), then we get subtraction of elements $(X_1, Y_1, Z_1, T_1) - (X_2, Y_2, Z_2, T_2) = (X_3, Y_3, Z_3, T_3)$. So, contrary to the exponentiation operation, where subtraction from the exponent (division) is an expensive operation, subtraction on twisted Edwards curves is performed as fast as addition. This allows to use the non-adjacent form (NAF) of a natural number to perform multiplication.

The non-adjacent form of a number is a signed-digit representation, in which non-zero digits cannot be adjacent. For example, non-adjacent form of 7 in signed binary numeral system is (1 0 0 -1). This definition can be strengthened by the concept of window, which gives the additional condition that only one of any consecutive W digits can be non-zero. Such a representation has the minimal Hamming weight among all representations with 2^{W-2} distinct digits. Fig. 2.5 shows the comparison of the sliding window and the non-adjacent form of width 4 for number $e = 11957708941720303968251$.

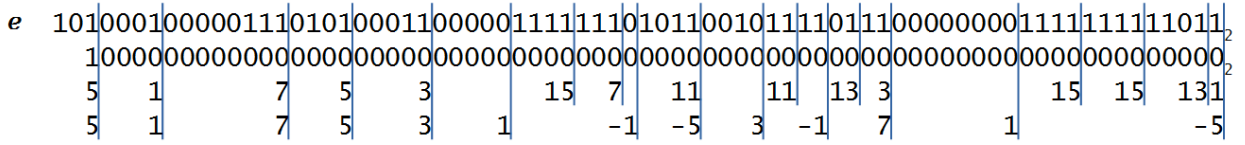


Figure 2.5: Sliding window $W = 4$ and NAF_4 .

The non-adjacent form NAF_W can be obtained with the following algorithm:

Algorithm 2.6: Non-adjacent form of width W of number e .

Output: $E_i : e = \sum E_i 2^i$

```

1   $i \leftarrow 0$ 
2  while  $e > 0$  do
3      if  $\text{odd}(e)$  then
4           $E_i \leftarrow e \bmod 2^W$ 
5          if  $E_i \geq 2^{W-1}$  then  $E_i \leftarrow E_i - 2^W$ 
6           $e \leftarrow e - E_i$ 
7      else
8           $E_i \leftarrow 0$ 
9           $e \leftarrow e/2$ 
10      $i \leftarrow i + 1$ 
11 end
```

The resulting representation E_i can be used in a way similar to e_i in the left-to-right exponentiation algorithm, with the corresponding multiplication operation addressing the precomputed dictionary. The dictionary is half the size of the one in the sliding window method. Or, from another angle, W can be increased by 1 at the same computational cost. Furthermore, the dictionary can be normalized, meaning the coordinates of each element can be divided by Z . There are algorithms that allow to perform normalization of multiple values while executing the extended Euclidean algorithm only once. The normalized dictionary saves one multiplication in (2.8.12). And since the doubling operation follows each addition operation but the last one, there is no need to compute T_3 in (2.8.12). With $a = 1$ and $W = 10$ –15, the multiplication of a group element by a natural number may cost as little as ~ 7.5 –7.8 multiplications per bit, which is significantly better than $\sim 1.5 \cdot 6 = 9$ multiplications per bit when using Montgomery curves with differential addition chains.

Operations on Edwards curves can employ several different methods of error checking. For example,

one can use the fact that single-coordinate and two-coordinate operations produce the same value of y (after normalization). Consider an algorithm requiring a lot of elements at a regular interval. The starting element can be computed with two-coordinate operations and non-adjacent form, then the elements at the regular interval are computed with the single-coordinate differential addition. The last element is checked by computing it again with two-coordinate operations and non-adjacent form.

Also, the two-coordinate operations can use the elliptic curve equation itself as an error checking condition

$$aX^2 + Y^2 \stackrel{?}{\equiv} Z^2 + dT^2 \pmod{m}. \quad (2.8.14)$$

One needs to store a and d to perform the check. But if a is fixed, like $a = 1$, it is possible to relatively easy calculate

$$d = \frac{aX^2 + Y^2 - Z^2}{T^2} \pmod{m} \quad (2.8.15)$$

and compare it with the original value. For the purpose of error checking, one can choose to store only a hash value of d instead of the whole number. If an arithmetic error happens, with a high probability it results in the element “jumping” from the original curve to another with a different set of parameters. It means both d and its hash change. (2.8.15) works as a signature of the curve.

Besides, since the two-coordinate operations on Edwards curves behave like multiplication and squaring of numbers, all algorithms described in this chapter can be implemented using elliptic curves.

Chapter 3

Factorization

Factoring big numbers is hard.

Acknowledgments

The author would like to thank:

Yves Gallot for opening my eyes on the algebraic side of FFT.

George Woltman for answering my many questions.

Darren Li for constructive advices and editing the English text.

Roman Trunov for supporting the practical implementation.

PrimeGrid community for trust and confidence.

my family for unconditional support.

sponsors for donating Monero to 453MFNnC7N7NK9iK99FoNh3xZ4wY9yqt1Qk6F
62iUZjg6bv6xD3oH3U2EABnUQMmco5BEK5FAhtKF18Mn43QQsysNHX9wB3.

Bibliography

- [1] D. H. Bailey. FFTs in External or Hierarchical Memory. *Supercomputing '89: Proceedings of the 1989 ACM/IEEE conference on Supercomputing*, pages 234–242, 1989. <https://doi.org/10.1145/76263.76288>.
- [2] Daniel J. Bernstein. Removing redundancy in high-precision Newton iteration, 1998. <https://cr.yp.to/papers.html#fastnewton>.
- [3] Richard Crandall and Barry Fagin. Discrete weighted transforms and large-integer arithmetic. *Mathematics of Computation*, 62, pages 305–324, 1994. <https://doi.org/10.1090/S0025-5718-1994-1185244-1>.
- [4] Colin Percival. Rapid multiplication modulo the sum and difference of highly composite numbers. *Mathematics of Computation*, 72, pages 387–395, 2003. <https://doi.org/10.1090/S0025-5718-02-01419-9>.
- [5] Pavel Atnashev and George Woltman. Edwards curves and FFT-based multiplication. Cryptology ePrint Archive, Report 2021/1061, 2021. <https://eprint.iacr.org/2021/1061>.
- [6] Christophe Doche. Exponentiation. *Handbook of Elliptic and Hyperelliptic Curve Cryptography*, pages 145–168, 2006. <http://hyperelliptic.org/HEHCC/chapters/chap09.pdf>.
- [7] Robert Gerbicz. Fast and robust error checking on Proth/Pepin tests, 2017. <https://www.mersenneforum.org/showthread.php?t=22510>.
- [8] Pavel Atnashev. Efficient Proth/PRP Test Proof Scheme, 2020. <https://www.mersenneforum.org/showthread.php?t=25323>.
- [9] Krzysztof Pietrzak. Simple Verifiable Delay Functions. Cryptology ePrint Archive, Report 2018/627, 2018. <https://eprint.iacr.org/2018/627>.
- [10] Darren Li and Yves Gallot. An Efficient Modular Exponentiation Proof Scheme. arXiv:2209.15623 [cs.CR], 2023. <https://arxiv.org/abs/2209.15623>.
- [11] Peter L. Montgomery. Evaluation of $V_n(P, 1)$ via Lucas chains, 1983. <https://cr.yp.to/bib/1992/montgomery-lucas.pdf>.
- [12] Daniel J. Bernstein and Tanja Lange. Montgomery curves and the Montgomery ladder. Cryptology ePrint Archive, Report 2017/293, 2017. <https://eprint.iacr.org/2017/293>.
- [13] Peter L. Montgomery. Speeding the Pollard and elliptic curve methods of factorization. *Mathematics of Computation*, 48, pages 243–264, 1987. <https://doi.org/10.1090/S0025-5718-1987-0866113-7>.
- [14] Daniel J. Bernstein and Tanja Lange. Faster addition and doubling on elliptic curves. Cryptology ePrint Archive, Report 2007/286, 2007. <https://eprint.iacr.org/2007/286>.

- [15] Huseyin Hisil, Kenneth Koon-Ho Wong, Gary Carter, and Ed Dawson. Twisted Edwards Curves Revisited. Cryptology ePrint Archive, Report 2008/522, 2008. <https://eprint.iacr.org/2008/522>.